

The Journal of Systems & Software

The Bottleneck Effect: When Small-Model Scaling Fails for Code Generation

--Manuscript Draft--

Manuscript Number:	
Article Type:	Research Paper
Keywords:	Large language models Code generation Software engineering Program synthesis Empirical software engineering Benchmarking Model scaling Neural code models
Corresponding Author:	Ashish Pandey Khwopa College of Engineering NEPAL
First Author:	Ashish Pandey
Order of Authors:	Ashish Pandey
Abstract:	Large language models have become increasingly capable at code generation, but it remains unclear when scaling alone improves executable-code reliability. This paper studies small decoder-only models across HumanEval, EvalPlus, MBPP, and LiveCodeBench-style evaluations, comparing general-language GPT-2 models with code-pretrained alternatives under fixed decoding and artifact-traceable evaluation. The results show that increasing parameter count within the GPT-2 family does not reliably improve code-generation success, while code-specific pretraining produces a much stronger executable-code profile. Additional robustness controls, error analyses, and targeted mechanistic probes indicate that the observed gap is not only a benchmark artifact but reflects differences in how models represent and preserve code-relevant structure. The study contributes an empirical and reproducibility-focused account of small-model code-generation failure modes, with processed outputs, scripts, and claim-to-artifact audits supplied for review.

May 12, 2026

Dear Editor-in-Chief,

Please consider our manuscript, "The Bottleneck Effect: When Small-Model Scaling Fails for Code Generation," for publication as a regular research article in the Journal of Systems and Software.

This paper studies a practical software-engineering problem: why small language models that appear similar in size can differ sharply in code-generation reliability. Instead of treating benchmark accuracy as a single leaderboard number, the manuscript traces failures across benchmarks, prompt formats, decoding choices, layer interventions, and artifact provenance.

The manuscript should fit JSS because it contributes empirical evidence about AI-based software generation systems, their evaluation reliability, and the engineering choices that shape whether generated code executes correctly. The main result is not a generic "bigger is better" scaling claim. The evidence shows that code-generation reliability depends on model family, code-specific pretraining, benchmark distribution, prompt interface, decoding policy, and the metric used to summarize generated programs.

The submission includes a reproducible artifact trail: processed result tables, run logs, plotting scripts, benchmark summaries, and figure-generation scripts. These artifacts are intended to make the reported claims auditable rather than dependent on manually transcribed results.

Thank you for considering the manuscript.

Sincerely,

Ashish Pandey
Department of Computer and Electronics Engineering
Khwopa College of Engineering, Nepal
ashishpanday9818@gmail.com

Highlights

- Small-model code generation does not improve reliably with parameter scaling alone.
- Code-specialized pretraining explains much more than GPT-2 scale in this regime.
- Layer ablations expose distinct syntax, runtime, and residual robustness profiles.
- Prompt format and decoding choices substantially change code-generation reliability.
- Reproducible artifacts link benchmark claims to saved outputs, logs, and scripts.

The Bottleneck Effect: When Small-Model Scaling Fails for Code Generation

Ashish Pandey^a

^a*Department of Computer and Electronics Engineering, Khwopa College of Engineering, Nepal*

Abstract

Scaling language models often improves next-token prediction, but code generation is an execution-constrained software-engineering task: a completion must parse, run, and satisfy tests. We study whether small-model code generation improves with parameter scaling, code-specialized pretraining, and more distributed layerwise computation. Across HumanEval, MBPP, EvalPlus HumanEval+/MBPP+, and LiveCodeBench release v2, GPT-2 Medium (355M) does not reliably improve over GPT-2 Small (124M). On HumanEval, GPT-2 Medium reaches 4.8% success versus 5.2% for GPT-2 Small, and the problem-level difference is not significant (+0.37 percentage points, 95% CI $[-0.73, 1.37]$, $p = 0.50$). On MBPP, both GPT-2 variants fall to 0.0% success. In contrast, the similarly sized CodeGen-350M reaches 37.4% success on HumanEval and 7.39% success / 22.96% pass@5 on MBPP, while retaining the only nonzero signal under stricter EvalPlus and LiveCodeBench scoring.

To test whether this gap is only a family comparison, we also evaluate the CodeGen-350M NL→Multi→Mono continued-pretraining ladder. Mono is strongest on both HumanEval and MBPP, and MBPP improves monotonically from 0.02% to 1.23% to 2.76%, showing that code- and Python-focused continued pretraining matters even when architecture and parameter count are held fixed. We then use full-layer ablation, graded ablation, failure-mode analysis, and activation steering to examine how these models organize executable-code behavior. GPT-2 Medium is globally brittle: every single-layer ablation drives success to 0%, but layer 12 uniquely shifts failures into a mixed syntax/runtime regime. A scaled-ablation pilot shows

Email address: ashishpanday9818@gmail.com (Ashish Pandey)

that partial weakening first causes syntax collapse, while full removal reopens runtime errors, suggesting a thresholded execution-relevant stage. CodeGen is still vulnerable to ablation, but it retains residual success across several layers; its layer-13 pilot preserves 28–30% success under partial attenuation and 24% even under full zeroing.

Finally, a five-dimensional GPT-2 Medium layer-12 probe reaches 72.5% held-out success-vs.-syntax accuracy, close to the 75.0% full-layer probe, and steering along this sparse direction doubles subset success from 6.0% to 12.0%. Matched random controls make this intervention promising but not definitive. Overall, in this small-model regime, the evidence is better explained by code-specific pretraining and the distribution of execution-relevant computation across depth than by parameter count alone.

Keywords: code generation, empirical software engineering, large language models, model evaluation, mechanistic interpretability, software reliability

1. Introduction

Neural language-model scaling reliably improves average loss Kaplan et al. (2020); Hoffmann et al. (2022), and larger code models can improve program-synthesis benchmarks Austin et al. (2021). But code generation is not only a next-token prediction problem: a completion must parse, execute, and satisfy tests. It is therefore not obvious that modest parameter scaling within a general-language model family should improve execution reliability once code-specific pretraining and layerwise organization differ.

We study that gap in the small-model regime. On the HumanEval Python programming benchmark Chen et al. (2021), GPT-2 Medium (355M parameters) reaches **4.8% success**, statistically indistinguishable from GPT-2 Small (124M parameters) at **5.2% success**. Yet CodeGen-350M Nijkamp et al. (2022)—with nearly identical parameter count to GPT-2 Medium—reaches **37.4% success**, a $7.8\times$ improvement. The same ordering persists on the full MBPP test split: both GPT-2 models fall to **0.0% success**, while CodeGen still reaches **7.39% success**. Under stricter EvalPlus re-scores, both GPT-2 variants remain at **0.0% pass@1** on HumanEval+ and **0.0% on pass@1 and pass@10** on MBPP+, while CodeGen retains **2.1% pass@1** on HumanEval+ and **1.1% / 6.4%** on MBPP+. A contamination-aware LiveCodeBench stress test is harsher for all three small models: GPT-2 Small and GPT-2 Medium both remain at **0.0%** on pass@1, pass@5, and

pass@10, while CodeGen is only marginally nonzero at **0.02%, 0.10%, and 0.20%**. Finally, a within-family CodeGen-350M continued-pretraining ladder (NL→Multi→Mono) shows that Mono is best on both HumanEval and MBPP, with MBPP becoming cleanly monotone and HumanEval remaining mixed at the intermediate Multi checkpoint. These results motivate a narrower question than whether scale helps in general: *when models are similarly small, what architectural properties let some of them organize executable-code behavior more effectively than larger general-language baselines?*

1.1. The Bottleneck Hypothesis

We hypothesize that code generation performance depends not on parameter count but on how models distribute code-relevant computation across layers. Specifically, we propose that models with brittle, sharply concentrated layerwise processing collapse under harsh interventions, while models with more distributed processing retain partial residual performance and more diverse failure signatures.

To test this hypothesis, we conduct systematic layer-wise ablation experiments: for each layer in each model, we zero out all activations and measure the impact on code generation success. This intervention reveals which layers are especially damaging when removed and how the resulting failure modes differ across architectures.

1.2. Research Questions and Evidence Map

We organize the study around four research questions. This framing is useful because code generation quality is not a single scalar property: the same model can differ in benchmark success, strict-test robustness, failure mode, and intervention sensitivity.

Table 1: Research Questions, Evidence, and Main Takeaways

Question	Evidence Used	Takeaway
RQ1: Does GPT-2 scaling help?	HumanEval, MBPP, EvalPlus, LiveCodeBench	No reliable GPT-2 Small→Medium gain.
RQ2: Does code pretraining matter?	GPT-2 vs. CodeGen; CodeGen ladder	Yes; code-specialized checkpoints are stronger.
RQ3: What changes under intervention?	Full/graded ablation and failure modes	GPT-2 Medium is brittle; CodeGen is more distributed.
RQ4: Is there a compact control signal?	Layer-12 probe and matched-control steering	Locally useful, but not yet definitive.

1.3. Key Findings

Through ablation of all 56 layers across three models (GPT-2 Small: 12 layers, Medium: 24 layers, CodeGen: 20 layers) evaluated on 16,300–16,400 code generation attempts, we discover:

1. **The GPT-2 scaling gap is numerically small on HumanEval and disappears on MBPP:** GPT-2 Small exceeds GPT-2 Medium by only 0.37 percentage points on HumanEval (95% CI $[-0.73, 1.37]$, $p = 0.50$), while both score 0.0% on full-coverage MBPP.
2. **GPT-2 Medium is globally fragile but has a distinctive mid-depth failure mode:** every GPT-2 Medium layer ablation reduces success to 0%, but ablating layer 12 uniquely converts 23.9% of failures from syntax errors into runtime errors, indicating a special role in producing executable-but-incorrect code.
3. **CodeGen exhibits broader residual robustness:** under the same harsh intervention, CodeGen retains nonzero success in four layers (5, 7, 13, 18), with layer 13 preserving 29.5% success rather than collapsing completely. Under graded attenuation, the same layer still retains 28–30% success at scales 0.75 and 0.5.
4. **A graded pilot sharpens the layer-12 interpretation:** on a 10-problem / 5-sample subset, scaling layer 12 from 1.0 to 0.5 and 0.25 first pushes GPT-2 Medium toward near-total syntax collapse, while only full zeroing reproduces the runtime-error regime (28.0%), indicating a thresholded execution checkpoint.
5. **A compact probe captures most of the layer-12 linear signal:** five dimensions (121, 346, 615, 820, 169) in GPT-2 Medium’s layer 12 achieve 72.5% held-out success-vs.-syntax classification accuracy, compared with 75.0% for the full 1024-dimensional layer.
6. **Constructive steering is useful but only suggestively specific:** steering GPT-2 Medium along the learned top-5 layer-12 vector raises subset success from 6.0% to 12.0% at $\alpha = 2.0$. Against 20 matched sparse random controls, the learned vector beats 19 and exceeds the control mean (6.2%), but one control reaches 14.0%, so the specificity evidence is promising rather than definitive.
7. **The scaling failure transfers across benchmarks and stricter tests:** on full-coverage MBPP (257 tasks, 20 samples each), both GPT-2 variants remain at 0.0% success, whereas CodeGen reaches

7.39% success and 22.96% pass@5 with highly significant pairwise gaps ($p = 0.0001$). Under a stricter HumanEval+ re-score on the first five saved samples per task, both GPT-2 variants stay at 0.0% pass@1 while CodeGen retains 2.1%. On contamination-aware LiveCodeBench release v2 Jain et al. (2024), both GPT-2 variants remain at 0.0% on pass@1, pass@5, and pass@10 and CodeGen remains only marginally nonzero, showing that all three small models are near-zero on a substantially harder modern benchmark.

8. **A within-family CodeGen ladder strengthens the pretraining argument:** in the 350M CodeGen NL→Multi→Mono continued-pretraining ladder, Mono is strongest on both benchmarks. On full-coverage HumanEval it reaches 39.0% success, significantly above both NL (31.60%) and Multi (30.66%), while NL vs. Multi is not significant. On MBPP the ladder is cleanly monotone (0.02%→1.23%→2.76%), with significant NL<Multi and Multi<Mono improvements. We therefore interpret the ladder as evidence about continued-pretraining plus curriculum effects rather than a perfectly factorized distribution-only ablation.
9. **Prompt and decoding controls expose systems-level bottlenecks:** targeted robustness controls show that prompt format and decoding temperature can change success substantially without changing model size. On HumanEval, CodeGen-Mono moves from 42.6% success under signature-only prompts to 71.2% under comment-plus-signature prompts. On MBPP, low temperature increases successful samples for both CodeGen-Mono and Qwen2.5-Coder, but reduces distinct-task coverage relative to standard decoding.
10. **Error profiles differ across checkpoints:** GPT-2 models produce 28.7% indentation errors and 8.1% keyword errors, while CodeGen produces 23.1% bracket errors and 7.4% indentation errors, revealing a distinct syntax-error profile without isolating the source of that difference.

1.4. Contributions

This work makes four primary contributions:

1. **Empirical:** We document and explain the scaling paradox in code generation through comprehensive evaluation on HumanEval, strict HumanEval+ and MBPP+ re-scoring with EvalPlus Liu et al. (2023), full-

coverage MBPP, and a contamination-aware LiveCodeBench stress test across three model scales.

2. **Methodological:** We combine sample-level evaluation with problem-level bootstrap confidence intervals, paired permutation tests, hard layer ablation, graded ablation, and matched-control activation steering to study code-generation failures from both statistical and mechanistic angles.
3. **Mechanistic:** We identify a compact layer-12 activation subspace that captures most of the linear probe performance, show that graded weakening of layer 12 reproduces the same execution-relevant transition seen under full ablation, and show that constructive steering along this subspace improves execution while yielding suggestive, but not yet definitive, evidence of specificity over matched sparse random controls.
4. **Architectural:** We provide evidence-based design principles for code-generating models: distributed processing across multiple critical layers prevents brittleness and enables graceful degradation, while code-specialized models preserve useful computation under graded interventions more effectively than general-language models. A within-family CodeGen continued-pretraining ladder further shows that architecture-matched checkpoints can still diverge substantially as code- and Python-specific pretraining accumulates.

These findings challenge the assumption that parameter scaling alone improves code generation, suggesting that in this small-model regime architectural choices during pretraining—particularly how computation is distributed across layers—are more informative than model size alone.

2. Related Work

2.1. Code Generation Models

Recent progress in neural code generation follows from scaling language models trained on code corpora. Codex Chen et al. (2021) demonstrated that GPT-3 (175B parameters) fine-tuned on GitHub code achieves 28.8% pass@1 on HumanEval. CodeGen Nijkamp et al. (2022) improved this through multi-turn program synthesis, achieving 29.3% with 16B parameters. AlphaCode Li et al. (2022) further advanced the field by generating multiple candidate solutions and filtering through test cases, though requiring massive compute.

However, these works focus on maximizing performance through scale, ensembling, and test-time compute, rather than understanding the internal mechanisms of code generation. Our work differs by investigating *why* some architectures succeed where others fail, even at comparable scale.

2.2. Evaluation Robustness in Code Generation

Functional-correctness benchmarks can overstate code quality when their tests are too sparse. EvalPlus addresses this by augmenting HumanEval and MBPP with much larger generated test suites, showing that stronger tests can expose previously accepted but incorrect programs and even change model rankings Liu et al. (2023). LiveCodeBench addresses a complementary threat: contamination and benchmark aging. By continuously collecting newer contest problems, it provides a harder time-aware evaluation surface beyond static suites such as HumanEval and MBPP Jain et al. (2024).

These benchmark developments motivate two design choices in our study. First, we retain the classical benchmarks because they support dense, reproducible mechanistic analysis and direct comparison with prior code-generation work Chen et al. (2021); Austin et al. (2021). Second, we add strict EvalPlus re-scoring and contamination-aware LiveCodeBench validation so the main ranking is not interpreted only through historically permissive tests or older static problems. Our contribution is not a new benchmark; it is a mechanistic account whose empirical claims are checked across increasingly strict evaluation surfaces.

2.3. Layer-wise Analysis of Transformers

Understanding which layers perform which computations has been studied extensively for natural language tasks. Clark et al. (2019) showed that different layers encode different linguistic properties, with early layers capturing syntax and later layers capturing semantics. Voita et al. (2019) demonstrated that layer representations can be interpreted through linear projections, revealing compositional structure.

For code models specifically, Ma et al. (2024) probe seven pretrained code models, including CodeBERT, and report that syntax is captured more consistently than semantics across the family. However, these analyses are observational (measuring what layers *do*) rather than interventional (measuring what layers are *necessary for*), limiting causal conclusions.

Our ablation methodology extends this line of work by using interventions (zeroing layer activations) to establish which layers are causally necessary for

successful generation, revealing bottlenecks invisible to observational analysis.

2.4. *Mechanistic Interpretability*

The mechanistic interpretability research program Olsson et al. (2022); Conmy et al. (2023) aims to reverse-engineer neural networks into human-understandable algorithms. Meng et al. (2022) identified specific attention heads and MLP layers responsible for factual recall, enabling targeted interventions. Vig et al. (2020) formalized methods for attributing model behavior to specific components through counterfactual interventions.

The circuits thread in mechanistic interpretability has successfully reverse-engineered small vision models, identifying exact algorithms for tasks like curve detection. For language models, prior work has identified induction heads (responsible for in-context learning) Olsson et al. (2022) and factual recall mechanisms Meng et al. (2022).

Our work applies mechanistic interpretability to code generation, identifying not just *which* layers matter but *how* their failure modes differ (syntax errors vs. runtime errors vs. timeouts), revealing the computational stages of program synthesis.

2.5. *Scaling Laws and Their Limits*

Neural scaling laws Kaplan et al. (2020); Hoffmann et al. (2022) predict that loss decreases smoothly with model size, dataset size, and compute. Program-synthesis work also reports broad scaling trends: Austin et al. (2021) find log-linear performance growth across 244M–137B models on MBPP. Those results establish that scale can help, but they do not imply that every modest within-family increase will improve execution reliability under every pretraining regime.

Recent work has also identified task-specific deviations from simple scaling narratives. Wei et al. (2022) showed that some reasoning tasks exhibit inverse scaling, contradicting a universal “larger is always better” assumption. Our study addresses a narrower regime: similarly small decoder-only models, where the key empirical contrast is not frontier-scale failure but the interaction between pretraining specialization and how executable-code behavior is distributed across depth.

3. Methodology

3.1. Models

We evaluate three decoder-only Transformer models Vaswani et al. (2017) representing different scales and pretraining regimes:

GPT-2 Small Radford et al. (2019) (124M parameters): 12 layers, 768 hidden dimensions, 12 attention heads. Pretrained on WebText (40GB diverse web text). Serves as our small-scale baseline.

GPT-2 Medium Radford et al. (2019) (355M parameters): 24 layers, 1024 hidden dimensions, 16 attention heads. Same pretraining as Small but larger capacity. Tests whether scaling improves code generation.

CodeGen-350M Nijkamp et al. (2022) (350M parameters): 20 layers, 1024 hidden dimensions, 16 attention heads. Pretrained sequentially on The Pile Gao et al. (2021) (natural language corpus including GitHub code), BigQuery (multilingual code in C, C++, Go, Java, JavaScript, Python), and BigPython (217.3 GiB of public Python code from GitHub repositories). Comparable parameter count to GPT-2 Medium but code-specialized pretraining.

All models use causal (autoregressive) attention with learned positional embeddings. We use pretrained checkpoints without fine-tuning to isolate the effect of base model architecture and pretraining distribution.

3.2. Evaluation Dataset

We evaluate on three benchmark families. The first is **HumanEval** Chen et al. (2021), a benchmark of 164 hand-written Python programming problems covering algorithms, data structures, and string manipulation. Each problem includes:

- A function signature and docstring describing the task
- Unit tests to verify correctness
- Diverse difficulty levels from simple (reverse a string) to complex (dynamic programming)

The second benchmark is the sanitized **MBPP** test split Austin et al. (2021), which contains 257 short Python programming tasks with reference tests derived from natural-language specifications. We use HumanEval for

the full mechanistic study and MBPP for external validation of the central scaling claim.

For HumanEval, we generate 100 candidate solutions per problem using temperature sampling (temp=0.8, top-p=0.95), yielding 16,400 intended completions per fully saved run. In the synced main CodeGen artifact, however, **HumanEval/129** still has no saved completions, so the scored sample count is 16,300 for that one condition. For the full MBPP validation, we generate 20 samples per task, yielding 5,140 attempts per model. This sampling strategy balances diversity (avoiding repetition) with quality (avoiding random text) while staying within the single-GPU compute budget.

As a stricter robustness check, we also re-score the first five saved HumanEval samples per problem with the HumanEval+ test augmentation distributed by EvalPlus Liu et al. (2023). We treat this as a completion-level pass@1 robustness check rather than a replacement for our main sample-level evaluation, because the stricter suite is used only on a capped subset of saved generations.

We additionally evaluate **MBPP+** with the official EvalPlus ‘mbpp’ evaluator Liu et al. (2023). We treat this as a separate versioned benchmark derived from MBPP-sanitized rather than as “MBPP with more tests,” and we report the Base+Extra score as the primary strict result. Our clean MBPP+run uses 378 EvalPlus tasks and 20 samples per task.

Our third benchmark family is the **LiveCodeBench** release v2 code-generation benchmark Jain et al. (2024). We use its default code-generation-lite configuration with 511 scored tasks and 10 samples per task as a contamination-aware stress test. Unlike HumanEval and MBPP, which are static curated Python suites, LiveCodeBench draws newer contest problems and is designed to make benchmark contamination harder. We therefore use it primarily to test whether the cross-model ordering survives on a modern harder benchmark, not to replace the mechanistic HumanEval analysis.

To partially separate pretraining-distribution effects from family differences, we also evaluate the **CodeGen-350M NL→Multi→Mono continued-pretraining ladder** Nijkamp et al. (2022). This keeps architecture and parameter count fixed while changing the added pretraining corpus and curriculum. We interpret it as a strong within-family control, but not as a matched-compute causal ablation, because later checkpoints receive additional tokens and optimization steps.

Coverage note: In the synced artifacts, the main CodeGen HumanEval

Table 2: Experimental Scope by Benchmark Family

Setting	Tasks	Samples / Task	Primary Role
HumanEval main	164 (GPT-2), 163 scored (CodeGen)	100	core comparison / pass@k
HumanEval+ strict	164	5	stronger test-suite check
MBPP full	257	20	external validation
MBPP+ strict	378	20	versioned strict validation
LiveCodeBench v2	511	10	contamination-aware stress test
CodeGen ladder (HumanEval)	164	100	within-family control
CodeGen ladder (MBPP)	257	20	within-family control
Mechanistic pilots	10	5	causal intervention tests

file still contains a zero-sample `HumanEval/129` entry. We therefore report sample-level main results over 163 scored CodeGen tasks, while the saved bootstrap summary retains the empty entry as a zero-success task when forming 164-task problem-level intervals.

3.3. Layer Ablation Protocol

To determine which layers are critical for code generation, we perform **systematic layer ablation**: for each layer l in each model, we:

1. **Generate solutions normally**: Run forward pass through all layers to obtain baseline success rate
2. **Ablate layer l** : Zero out all hidden activations at layer l during generation:

$$h_l^{(t)} \leftarrow \mathbf{0} \quad \forall t \in \{1, \dots, T\} \quad (1)$$

where $h_l^{(t)}$ is the hidden state at layer l and position t

3. **Generate with ablation**: Complete generation using ablated model
4. **Measure impact**: Compare success rate with vs. without ablation

We ablate each layer independently (not cumulatively) to isolate individual layer contributions. For each ablation condition, we test 50 randomly sampled problems with 20 generations each (1,000 samples per layer), balancing statistical power with computational cost.

Critical layer definition: A layer is considered *critical* if its ablation reduces success rate by $>50\%$ or changes the primary error mode (e.g., syntax $\rightarrow$ runtime errors).

3.4. Intervention Formalism

Let $h_l^{(t)} \in \mathbb{R}^{d_l}$ denote the residual stream after layer l at token position t . Our interventions operate directly on this representation during generation.

Full ablation zeroes the layer output:

$$\mathcal{A}_l(h_l^{(t)}) = \mathbf{0}. \quad (2)$$

Scaled ablation attenuates the same layer by a coefficient $\alpha \in [0, 1]$:

$$\mathcal{A}_{l,\alpha}(h_l^{(t)}) = \alpha h_l^{(t)}. \quad (3)$$

Activation steering adds a contrast vector v with strength β :

$$\mathcal{S}_{l,\beta,v}(h_l^{(t)}) = h_l^{(t)} + \beta v. \quad (4)$$

For GPT-2 Medium layer 12, we define

$$v = P_k(\mu_{\text{success}} - \mu_{\text{syntax}}), \quad (5)$$

where μ_{success} and μ_{syntax} are class-conditional mean activations and P_k keeps only the top- k discriminative coordinates.

To test specificity rather than generic perturbation sensitivity, we also construct matched sparse random controls $v^{(r)}$ by placing the same nonzero coefficient set on randomly chosen non-target dimensions while preserving sparsity and overall vector scale. If the learned subspace is mechanistically meaningful, steering with v should systematically outperform this matched-control distribution rather than merely improve relative to baseline once.

3.5. Problem-Level Statistical Validation

Sample-level percentages are useful descriptive summaries, but the independent unit of evaluation in HumanEval is the problem rather than the completion. We therefore complement raw percentages with problem-level statistics. For each model and task, we compute the per-problem success rate and the HumanEval pass@10 estimate Chen et al. (2021). We then draw 2,000 bootstrap resamples over tasks to obtain 95% confidence intervals for each model and use paired sign-flip permutation tests across shared tasks for model comparisons.

This distinction matters for the scaling claim. The raw success rates differ only slightly between GPT-2 Small and GPT-2 Medium, so a problem-level test is necessary to determine whether that gap reflects a stable effect or sampling noise. In contrast, the much larger CodeGen advantage should remain visible under either summary.

3.6. Reproducibility and Compute Constraints

All experiments use fixed pretrained checkpoints, fixed decoding hyperparameters (temperature 0.8, top- p 0.95), and deterministic seed schedules tied to the problem index and intervention condition. The main HumanEval sweeps are evaluated from saved JSON outputs, while the graded-ablation and activation-steering pilots are rerun directly under the same executor and timeout settings. The curated supplementary artifact archive includes the benchmark configs, figure-generation scripts, and saved artifact summaries used to regenerate every reported table and figure in this manuscript, with the main CodeGen HumanEval coverage caveat noted above.

The mechanistic follow-up experiments are intentionally smaller because they run with batch size 1 on a single 12 GB consumer GPU. For that reason, we treat the 10-problem / 5-sample intervention studies as targeted causal probes rather than replacements for the full benchmark. All tables and figures in this paper are generated from saved artifacts rather than manually transcribed counts.

3.7. Code Execution and Error Categorization

We execute each generated solution in an isolated Python sandbox with 5-second timeout and measure outcomes:

Success: All unit tests pass without errors

Syntax Error: Code fails to parse (invalid Python)

Runtime Error: Code parses but raises exception during execution (e.g., `IndexError`, `KeyError`, `TypeError`)

Timeout: Execution exceeds 5-second limit (typically infinite loops)

For syntax errors, we subcategorize into:

- **Indentation errors:** Incorrect whitespace (most common GPT-2 failure)
- **Bracket errors:** Mismatched parentheses, braces, brackets
- **Quote errors:** Unclosed or mismatched string delimiters
- **Keyword errors:** Invalid Python keywords or incorrect usage
- **Colon errors:** Missing colons after `if/for/while/def`
- **Other:** Rare syntax failures not fitting above categories

This fine-grained taxonomy reveals which models struggle with which syntactic constructs, providing insight into learned representations.

3.8. Activation Analysis

To understand *how* failure-mode transitions are encoded, we analyze activation patterns in GPT-2 Medium’s layer 12, the only ablation condition that substantially shifts failures from syntax errors to runtime errors.

For 200 randomly sampled generations (100 successful, 100 syntax-error), we:

1. Extract mean-pooled hidden states $h_{12} \in \mathbb{R}^{1024}$ from transformer block 12
2. Compute difference vectors: $\Delta h = h_{12}^{\text{success}} - h_{12}^{\text{syntax}}$
3. Identify dimensions with the largest magnitude differences
4. Train held-out logistic regression probes on the top-2, top-5, and full-layer activations
5. Reuse the top-5 contrast vector as a constructive steering intervention with $\beta \in \{-2, -1, 1, 2\}$ and compare it against matched sparse random controls

This analysis tests both whether a compact linear readout exists in the layer’s activation space and whether the same compact subspace can be used constructively at generation time. We still do not claim that any single dimension is causally sufficient on its own.

4. Results

4.1. The Scaling Paradox: Medium Performs Worse Than Small

Figure 1 summarizes the benchmark-wide pattern, while Table 3 presents the full HumanEval error-category totals:

Key observations:

1. **Scaling does not yield a reliable gain:** GPT-2 Medium (355M parameters) achieves 4.8% success versus GPT-2 Small (124M parameters) at 5.2% success. This small numerical reversal is not statistically reliable, but it does contradict the simple expectation that added parameters should monotonically improve code-generation success.

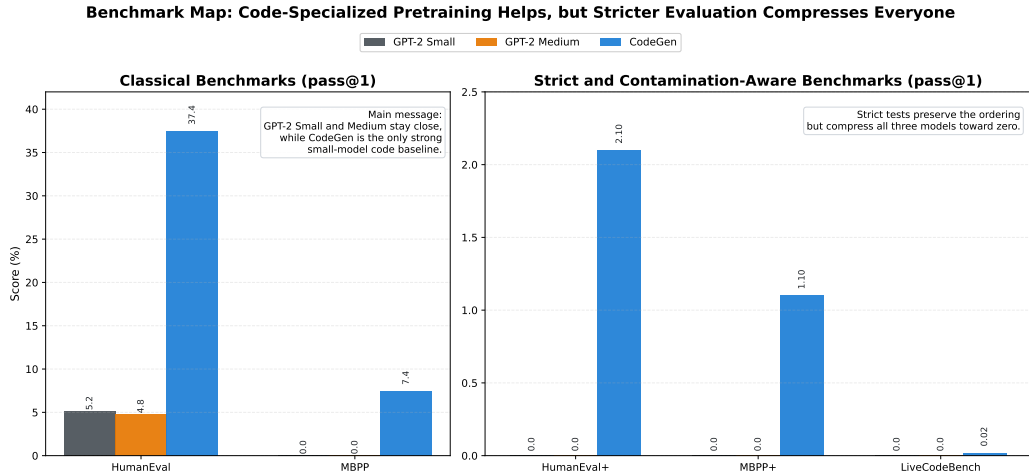


Figure 1: Benchmark-wide comparison across classical, strict, and contamination-aware evaluations. HumanEval and MBPP show the main scale-versus-pretraining gap: GPT-2 Small and GPT-2 Medium remain close, while CodeGen is far stronger. Stricter benchmarks preserve the ranking but compress all three models sharply toward zero.

Table 3: Model Performance on HumanEval (100 samples per problem; GPT-2 totals 16,400 attempts, while CodeGen has 16,300 scored attempts because HumanEval/129 has no saved completions)

Model	Params	Success	Syntax	Runtime	Timeout
GPT-2 Small	124M	5.2	94.4	0.4	0.0
GPT-2 Medium	355M	4.8	94.9	0.3	0.0
CodeGen-350M	350M	37.4	61.1	1.1	0.3

- Pretraining explains more than scale in this comparison:** CodeGen-350M has nearly identical parameter count to GPT-2 Medium but achieves $7.8\times$ higher success (37.4% vs. 4.8%), showing that parameter count alone does not explain the gap and that pretraining regime is the stronger explanation in this comparison Nijkamp et al. (2022).
- Failure modes differ across checkpoints:** GPT-2 models fail almost entirely on syntax (94.4-94.9% of all generations), while CodeGen leaves a larger fraction of generations executable enough to surface runtime errors and occasional timeouts, suggesting a different executable-code profile in this comparison.

Table 4: Problem-Level Statistical Validation

Model	Success % [95% CI]	pass@10 [95% CI]
GPT-2 Small	5.15 [4.49, 5.84]	37.17 [33.50, 40.83]
GPT-2 Medium	4.79 [3.94, 5.77]	33.09 [29.55, 36.79]
CodeGen-350M	37.42 [35.77, 39.13]	97.90 [97.41, 98.36]

4. **Both GPT-2 scales fail similarly:** The near-identical error distributions between Small (94.4% syntax) and Medium (94.9% syntax) suggest they learned similar—and similarly inadequate—code representations despite $2.9\times$ parameter difference.

4.2. Statistical Validation of the Scaling Paradox

The raw percentages in Table 3 could still reflect task-level sampling noise, especially for the narrow 5.2% vs. 4.8% GPT-2 comparison. We therefore summarize problem-level bootstrap intervals and HumanEval pass@10 estimates in Table 4.

Finding: The GPT-2 Small vs. GPT-2 Medium difference is not statistically reliable at the problem level (+0.37 percentage points, 95% CI $[-0.73, 1.37]$, $p = 0.5001$). By contrast, both GPT-2 models differ dramatically from CodeGen: GPT-2 Small vs. CodeGen is -32.27 points (95% CI $[-34.06, -30.47]$, $p = 0.0001$), and GPT-2 Medium vs. CodeGen is -32.63 points (95% CI $[-34.57, -30.68]$, $p = 0.0001$).

Interpretation: The scaling paradox within the GPT-2 family is therefore best understood as a *failure to improve*, not as a confidently estimated reversal in favor of GPT-2 Small. The code-pretraining gap, however, is large, stable, and statistically unambiguous. Figure 2 makes the same contrast visible across both HumanEval and MBPP. For transparency, the saved bootstrap summary keeps all 164 CodeGen task IDs by treating the empty HumanEval/129 entry as zero success, whereas the sample-level table above is based on the 163 tasks with scored completions; Appendix Figure A.14 shows the final scored-vs-intended completion counts and the pre-sync repair burden for the repaired HumanEval artifacts.

4.3. External Validation on MBPP

To test whether the scaling paradox is specific to HumanEval, we reran all three models on the full sanitized MBPP test split, using all 257 tasks and 20

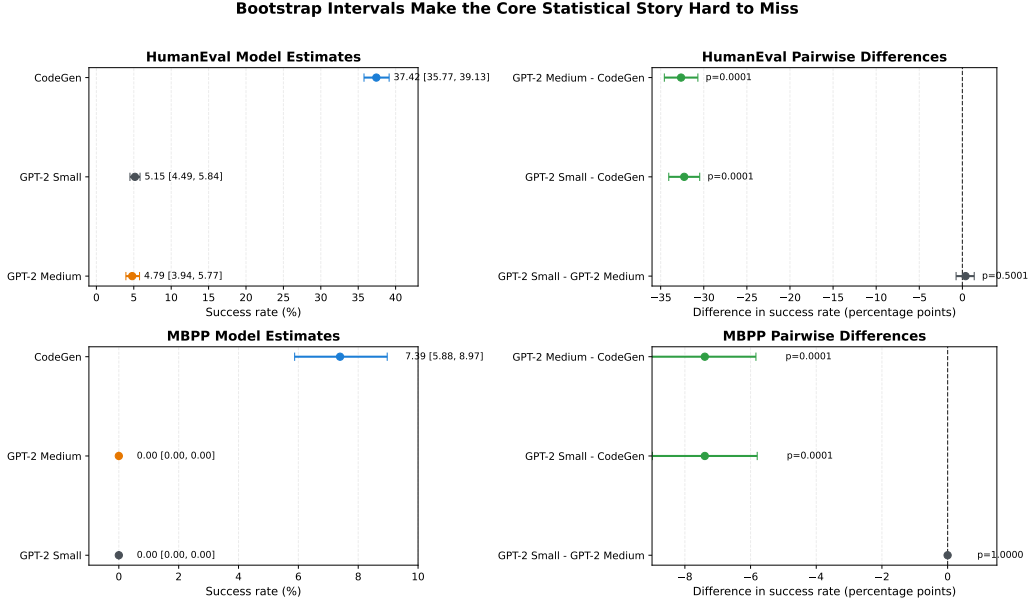


Figure 2: Bootstrap confidence intervals and pairwise success-rate deltas for the main HumanEval and MBPP comparisons. The GPT-2 Small / Medium gap overlaps zero on both benchmarks, whereas both GPT-2 models are cleanly separated from CodeGen.

Table 5: Full-Coverage MBPP Validation

Model	Success % [95% CI]	pass@5 [95% CI]	Syntax %	Runtime %
GPT-2 Small	0.00 [0.00, 0.00]	0.00 [0.00, 0.00]	96.30	0.54
GPT-2 Medium	0.00 [0.00, 0.00]	0.00 [0.00, 0.00]	98.31	0.18
CodeGen-350M	7.39 [5.88, 8.97]	22.96 [18.95, 27.09]	39.88	16.11

samples per task. This gives a second benchmark with different prompt style and shorter programming tasks, while keeping the same Python execution setting.

Finding: The scaling failure transfers cleanly. On MBPP, GPT-2 Small and GPT-2 Medium are tied at 0.0% success, while CodeGen reaches 7.39% success and 22.96% pass@5. The pairwise bootstrap differences against CodeGen remain highly significant for both GPT-2 Small and GPT-2 Medium (-7.39 points in each case, $p = 0.0001$), whereas GPT-2 Small vs. GPT-2 Medium is exactly 0.0 points ($p = 1.0$).

Interpretation: This external validation matters because it shows that the central claim is not a HumanEval artifact. Across both bench-

Table 6: Strict HumanEval Re-scoring with HumanEval+

Model	HumanEval pass@1	HumanEval+ pass@1
GPT-2 Small	0.00	0.00
GPT-2 Medium	0.00	0.00
CodeGen-350M	2.90	2.10

marks, scaling the general-language GPT-2 family fails to produce useful code-generation gains, while code-specific pretraining continues to produce a qualitatively different failure profile. MBPP makes this difference even starker: the GPT-2 models collapse almost entirely to syntax errors, whereas CodeGen preserves a much larger executable regime with mixed wrong-output and runtime failures.

4.4. Strict Re-Evaluation on HumanEval+

To test whether the main HumanEval ranking survives a stronger unit-test suite, we re-scored the first five saved HumanEval samples per task with HumanEval+, using the EvalPlus test augmentation. This gives a stricter completion-level pass@1 estimate without rerunning the full generation pipeline.

Finding: The stricter re-evaluation preserves the same ordering as the main benchmark comparison. Both GPT-2 variants remain at 0.0% pass@1, while CodeGen retains nonzero performance even after the extra hidden tests reduce it from 2.9% to 2.1%.

Interpretation: This result matters because it rules out an easy objection that our main HumanEval ranking is driven only by weak original unit tests. The absolute pass@1 values are lower here because HumanEval+ is stricter and because we cap the re-score at five saved samples per task, but the relative conclusion is unchanged: scaling GPT-2 does not help, whereas code-specialized pretraining still produces measurably more robust completions.

4.5. Strict Re-Evaluation on MBPP+

We next evaluate MBPP+ with the official EvalPlus ‘mbpp’ evaluator, reporting the Base+Extra score as our primary strict metric. Because MBPP+

Table 7: Strict MBPP+ Re-scoring with EvalPlus

Model	Tasks	MBPP+ pass@1	MBPP+ pass@10
GPT-2 Small	378	0.00	0.00
GPT-2 Medium	378	0.00	0.00
CodeGen-350M	378	1.10	6.40

Table 8: LiveCodeBench release v2 Validation

Model	Tasks scored	pass@1 %	pass@5 %	pass@10 %
GPT-2 Small	510	0.00	0.00	0.00
GPT-2 Medium	511	0.00	0.00	0.00
CodeGen-350M	511	0.02	0.10	0.20

is a versioned benchmark derived from MBPP-sanitized rather than the original MBPP split, we report it separately from Table 5 and do not compare it directly to legacy MBPP numbers.

Finding: The stricter MBPP+ evaluation preserves the same qualitative ordering as the full MBPP benchmark. Both GPT-2 variants remain at 0.0% under the stronger Base+Extra score, while CodeGen retains 1.1% pass@1 and 6.4% pass@10.

Interpretation: MBPP+ strengthens the negative claim and slightly narrows the positive one. The GPT-2 scaling failure survives a stricter, versioned test suite, while CodeGen remains the only model with nonzero strict performance. At the same time, the lower absolute numbers emphasize that even CodeGen’s advantage on small-model code generation is fragile under stronger hidden tests.

4.6. Contamination-Aware Validation on LiveCodeBench

To test whether the same ranking survives on a newer contamination-aware benchmark, we evaluated all three models on LiveCodeBench release v2 Jain et al. (2024) in its default code-generation-lite setting, using 10 samples per task. This benchmark is substantially harder than HumanEval or MBPP for models in the 124M–355M range.

Finding: The relative ordering is preserved, but the absolute performance landscape changes sharply. GPT-2 Small and GPT-2 Medium remain exactly at zero, while CodeGen is only marginally nonzero. For GPT-2 Small,

Table 9: CodeGen-350M Continued-Pretraining Ladder

Checkpoint	HumanEval tasks	HumanEval success %	MBPP success %
CodeGen-NL	164	31.60	0.02
CodeGen-Multi	164	30.66	1.23
CodeGen-Mono	164	39.00	2.76

one task did not yield a valid scored record under the custom export path, so pass@k is reported over 510 scored tasks; the other two models are scored over the full 511 tasks.

Interpretation: LiveCodeBench is therefore best read as a calibration result rather than a new positive benchmark win. It strengthens the negative claim that scale alone does not induce useful code generation in small general-language models, while also showing that a 350M code-pretrained model is still far from practically capable on a harder contamination-aware benchmark. The main positive CodeGen advantage is strongest on the classical Python benchmarks (HumanEval and MBPP), not on this harsher stress test. Appendix Figure A.13 shows the same compression as a pass@1 cascade across benchmark families.

4.7. Within-Family Continued-Pretraining Control in CodeGen

To reduce the objection that GPT-2 versus CodeGen is “just a family difference,” we evaluate the CodeGen-350M NL→Multi→Mono checkpoint ladder described by Nijkamp et al. (2022). This comparison holds architecture and parameter count fixed while changing the added continued-pretraining corpus. We treat it as a strong within-family control, not as a pure distribution-only isolation, because later checkpoints also receive additional optimization steps and tokens.

Finding: Mono is strongest on both benchmarks, but the ladder behaves differently across them. On full-coverage HumanEval, Mono significantly exceeds both NL (39.00% vs. 31.60%, $p = 0.0001$) and Multi (39.00% vs. 30.66%, $p = 0.0001$), while NL vs. Multi is small and not significant (31.60% vs. 30.66%, $p = 0.4421$). On MBPP, the ladder remains cleanly monotone: NL reaches 0.02%, Multi 1.23%, and Mono 2.76%, with significant improvements from NL to Multi ($p = 0.0001$) and Multi to Mono ($p = 0.0157$).

Interpretation: This is now stronger within-family evidence because the HumanEval ladder is full-coverage rather than OOM-truncated. Even

Within-Family CodeGen Ladder: Mono Is Best, but HumanEval Is Not Perfectly Monotone

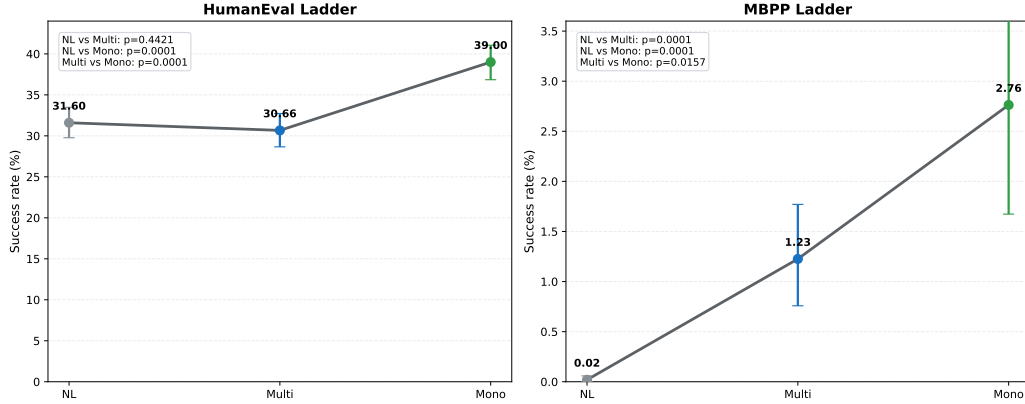


Figure 3: Within-family CodeGen ladder performance on HumanEval and MBPP. Mono is best on both benchmarks, but only MBPP is cleanly monotone at the intermediate Multi checkpoint. HumanEval improves decisively only at the final Mono stage.

so, the HumanEval ladder is still mixed at the intermediate checkpoint because Multi does not improve over NL, and the checkpoints still differ in additional compute and curriculum. We therefore use the ladder to support a continued-pretraining plus curriculum claim, not a matched-compute law. The MBPP result is cleaner and supports the narrower claim that continued-pretraining plus curriculum effects materially improve small-model code generation within a single family.

As an auxiliary diagnostic, prompt perplexity on the ladder prompts is lowest for Mono on both HumanEval (4.87) and MBPP (4.46), intermediate for NL (4.93 and 4.64), and highest for Multi (5.54 and 5.23). Following CodeGen, we interpret these numbers only as prompt-checkpoint compatibility diagnostics, not as direct explanations of functional gains, so they remain secondary evidence rather than a main result.

Figure 3 visualizes the key asymmetry: HumanEval only improves decisively at the final Mono stage, whereas MBPP improves throughout the ladder.

4.8. Prompt and Decoding Robustness Controls

The preceding results show that parameter count and pretraining distribution are not interchangeable. We next ask a more systems-oriented question: how much of the observed reliability depends on the surrounding genera-

tion protocol? We therefore ran targeted robustness controls on the three most informative small models: GPT-2 Medium, CodeGen-Mono 350M, and Qwen2.5-Coder 0.5B. These controls are not intended to replace the larger 100-sample main sweeps; they are sensitivity analyses using fixed saved artifacts, 20 samples per HumanEval task, and 10 samples per MBPP task.

Figure 4 summarizes the result. Prompt format has a large effect on HumanEval for code-specialized models: CodeGen-Mono moves from 42.6% sample success under signature-only prompting to 71.2% under comment-plus-signature prompting, while Qwen2.5-Coder moves from 41.6% under canonical prompting to 54.7% under comment-plus-signature prompting. GPT-2 Medium remains weak across all prompt styles, ranging only from 1.1% to 2.1% sample success, so prompt format does not rescue the general-language model’s syntax bottleneck.

Decoding temperature has a different profile. On HumanEval, GPT-2 Medium improves slightly at high temperature (3.1% vs. 2.1% standard), but remains far below the code-specialized models. CodeGen-Mono is strongest at standard temperature (56.1% sample success), and Qwen2.5-Coder is slightly strongest at low temperature (42.2%) but loses distinct-task coverage relative to standard decoding. On MBPP, low temperature improves sample success for both CodeGen-Mono (14.4% vs. 8.6% standard) and Qwen2.5-Coder (28.8% vs. 19.0% standard), but it reduces distinct-task coverage: CodeGen-Mono falls from 34.2% covered tasks under standard decoding to 30.7% under low temperature, and Qwen2.5-Coder falls from 54.9% to 49.8%.

Interpretation: These controls strengthen the systems framing. The bottleneck is not parameter count alone, and not even pretraining alone. Code-generation reliability also depends on prompt interface, decoding policy, benchmark distribution, and the metric chosen to summarize output. MBPP makes the tradeoff especially clear: an intervention can increase the number of successful samples while reducing the number of distinct problems for which the model finds any solution.

4.9. Scaled Follow-Ups on CodeGen Ladder Signature Layers

To test whether the ladder’s signature layers also show graded behavior, we ran a final 20-problem / 10-sample scaled-ablation follow-up on CodeGen-NL layer 11, CodeGen-Multi layer 7, and CodeGen-Mono layer 13 using scales {0.75, 0.5, 0.25, 0.0}.

Finding: Two of the three ladder follow-ups are mechanistically useful. In CodeGen-NL, layer 11 degrades in the expected direction under

Robustness Controls: Prompt and Decoding Choices Change the Bottleneck

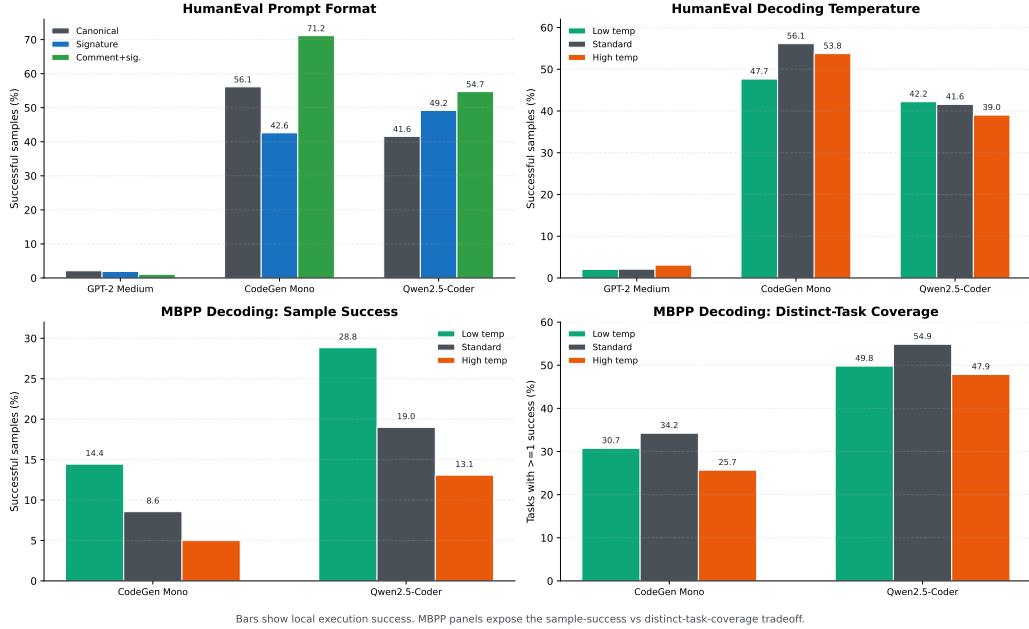


Figure 4: Targeted prompt and decoding robustness controls. Prompt format strongly changes HumanEval sample success for CodeGen-Mono and Qwen2.5-Coder while leaving GPT-2 Medium near zero. Decoding temperature is model- and benchmark-specific. On MBPP, low temperature increases repeated sample success but reduces distinct-task coverage, showing that aggregate success and reliability across tasks can move in opposite directions.

attenuation (25.5% $\rightarrow$ 22.5% $\rightarrow$ 16.0% $\rightarrow$ 8.5%), indicating that the earlier checkpoint relies on a comparatively brittle syntactic support layer. In CodeGen-Mono, layer 13 preserves 33.5% success at scale 0.75, falls to 2.0% at scale 0.25, and still retains 29.0% success under full zeroing while shifting 22.5% of outputs into runtime errors, reinforcing the main claim that later code-pretrained checkpoints retain a broader executable-code regime. CodeGen-Multi is less clean: layer 7 behaves plausibly under partial attenuation, but full zeroing unexpectedly improves success to 61.5%, so we treat that leg as anomalous and exclude it from the mechanistic claim.

Interpretation: These follow-ups strengthen the ladder interpretation without making it artificially cleaner than the evidence. The NL checkpoint looks more like a syntax-dependent intermediate stage, while the Mono

Table 10: Scaled Follow-Ups on CodeGen Ladder Signature Layers

Checkpoint	Layer	Baseline	0.25 scale	0.0 scale
CodeGen-NL	11	25.5	8.5	17.0
CodeGen-Multi	7	41.0	1.0	61.5
CodeGen-Mono	13	31.0	2.0	29.0

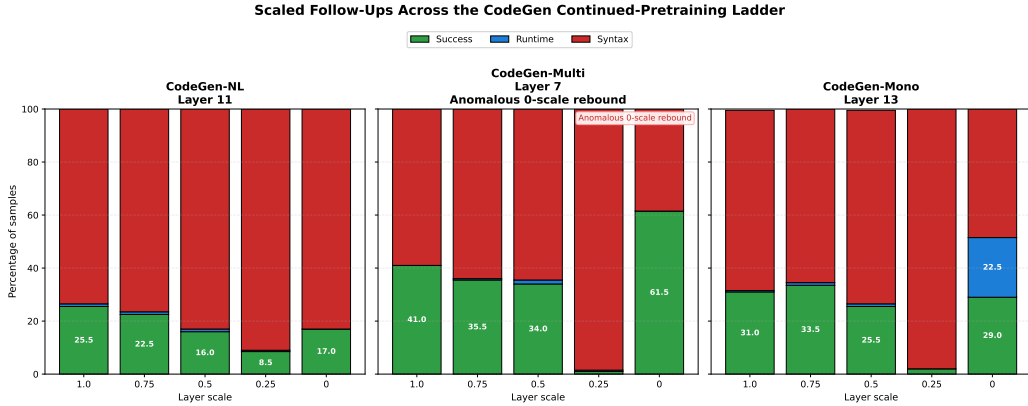


Figure 5: Scaled follow-up interventions on signature layers from the CodeGen continued-pretraining ladder. CodeGen-NL layer 11 degrades toward syntax collapse under attenuation, CodeGen-Mono layer 13 reopens a mixed executable/runtime regime under full zeroing, and CodeGen-Multi layer 7 is anomalous at 0.0 scale. We therefore use the NL and Mono legs as the clean within-family mechanistic evidence.

checkpoint again shows the mixed residual/runtime behavior that marks distributed executable-code support in the main CodeGen analysis. The anomalous Multi zero-ablation result prevents us from claiming a simple monotone mechanistic progression across the whole ladder, but it does not erase the broader pattern that later continued-pretraining stages can change both benchmark performance and the layerwise failure geometry of code generation.

4.10. Layer Ablation Reveals Distinct Failure Profiles

To understand *why* scaling fails for GPT-2 but succeeds for CodeGen, we ablate each layer individually and measure the impact on code generation success.

Key findings:

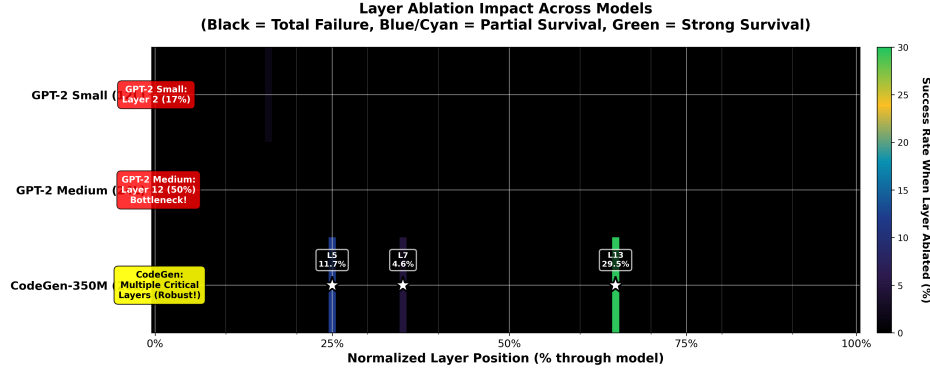


Figure 6: Single-layer zeroing across the three models. GPT-2 Medium falls to 0% success for every layer, with layer 12 standing out only through a runtime-error shift, whereas CodeGen retains several nonzero layers. The contrast is narrow brittleness versus broader residual support, not robustness versus no effect.

Table 11: Signature Ablation Layers and Failure Modes

Model	Total Layers	Signature Layer(s)	Residual Success	Failure Signature (Ablated)
GPT-2 Small	12	Layer 2 (17%)	1.8%	98.2% syntax
GPT-2 Medium	24	Layer 12 (50%)	0.0%	76.1% syntax, 23.9% runtime
CodeGen-350M	20	Layers 5, 7, 13, 18	11.7%, 4.6%, 29.5%, 0.1%	Mostly syntax; layer 13 adds 18.9% runtime

- GPT-2 Medium is uniformly brittle under full-layer ablation:** every single-layer ablation reduces success to 0%, so the model does not exhibit residual robustness anywhere in depth.
- Layer 12 is distinguished by failure mode, not residual success:** when layer 12 is ablated, GPT-2 Medium produces 23.9% runtime errors instead of collapsing entirely to syntax errors. No other medium layer shows this pattern.
- CodeGen retains partial survival in several layers:** layers 5, 7, 13, and 18 all preserve nonzero success when ablated, with layer 13 preserving 29.5% success. This is still a harsh failure regime, but it is broader than either GPT-2 variant.
- Residual robustness and signature depth move together in this three-model snapshot:** GPT-2 Small’s only surviving layer appears early (17%), GPT-2 Medium’s distinctive runtime-shift layer appears at mid depth (50%), and CodeGen’s strongest residual layer appears later (65%). We treat this as descriptive evidence rather than a general law.

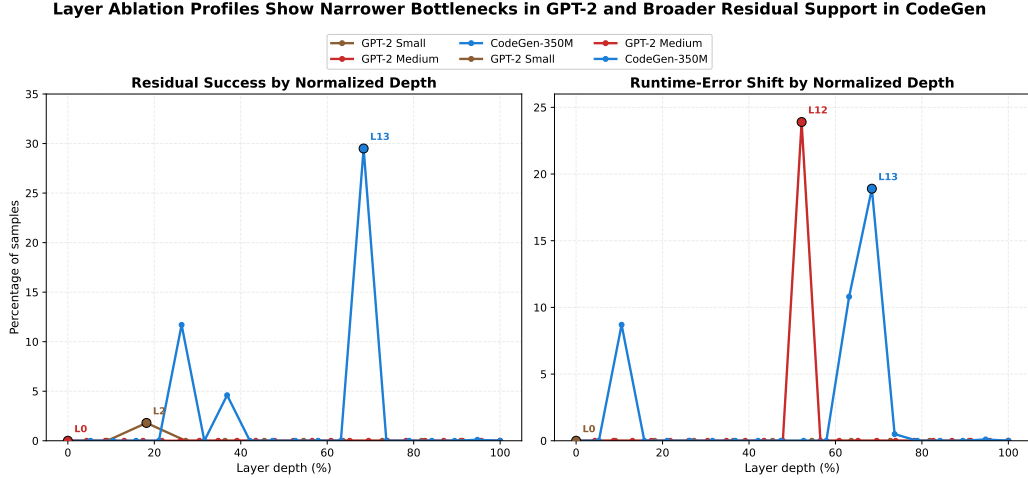


Figure 7: Normalized-depth view of the same ablations. GPT-2 Medium shows a narrow mid-depth runtime-shift spike without residual success, while CodeGen preserves both later residual success and a mixed executable/runtime regime centered on layer 13. This view isolates the mechanistic contrast more clearly than the heatmap alone.

Table 12: Syntax Error Distribution by Type (% of all syntax errors)

Model	Ind.	Brack.	Quote	Kwd	Colon	Other
GPT-2 Small	22.0	6.5	7.9	8.6	0.3	54.4
GPT-2 Medium	28.7	3.0	8.0	8.1	0.3	51.7
CodeGen-350M	7.4	23.1	5.3	2.4	4.7	57.0

4.11. Syntax Error Breakdown Reveals Distinct Failure Profiles

To compare syntax-error profiles across checkpoints, we categorize the types of syntax errors each model produces.

Key patterns:

1. **GPT-2 struggles with indentation:** Both GPT-2 models produce 22-29% indentation errors—far more than CodeGen (7.4%). This pattern is consistent with weaker indentation handling in the two GPT-2 checkpoints under our evaluation, but our experiments do not isolate why that difference arises.
2. **Medium’s indentation errors increase with scale:** GPT-2 Medium (28.7%) produces more indentation errors than Small (22.0%), showing that the larger GPT-2 checkpoint does not reduce this failure mode in our setting.

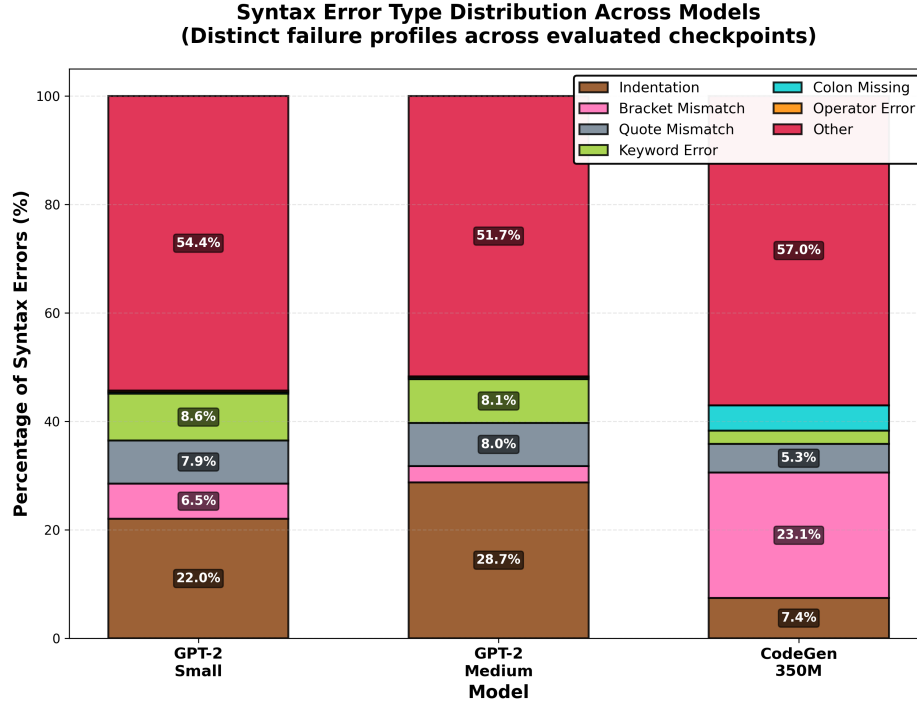


Figure 8: Syntax-error profile by model. GPT-2 errors are dominated by indentation, whereas CodeGen shifts toward bracket mismatches. The difference is consistent with distinct syntactic priors across the evaluated checkpoints, but the error profile alone does not identify the training-data cause.

3. **CodeGen fails on brackets:** CodeGen produces 23.1% bracket errors—more than $7\times$ GPT-2 Medium’s 3.0%. One plausible interpretation is that its broader code pretraining, which includes substantial Java and C++ proportions Nijkamp et al. (2022), shapes this profile. Our experiments, however, do not isolate training-data composition as the cause of the bracket-error gap.
4. **Keyword and quote errors are similar:** All models produce 5-8% quote errors and 2-9% keyword errors, suggesting that these categories are not the main source of the observed cross-model contrast in this sample.
5. **“Other” errors dominate:** 52–60% of syntax errors fall outside our six main categories, indicating long-tailed failure modes (invalid operators, malformed imports, incorrect statement structure) that remain outside

Table 13: Top Discriminative Dimensions in Layer 12

Dim	Mean (S)	Mean (F)	Diff.	% Sig.
121	−35.27	−29.54	5.73	0.90%
346	17.03	14.19	2.84	0.44%
615	6.96	4.21	2.75	0.43%
820	−3.15	−0.76	2.39	0.37%
169	−3.36	−1.02	2.34	0.37%
Remaining 1019 dims	—	—	—	97.49%

this coarse taxonomy.

4.12. Activation Analysis: A Compact Probe Signal at Layer 12

To understand the mechanistic basis of GPT-2 Medium’s distinctive layer-12 failure mode, we analyze which activation dimensions discriminate between successful and syntactically invalid generations.

Methodology: For 200 balanced samples (100 success, 100 syntax error), we mean-pool the hidden states $h_{12} \in \mathbb{R}^{1024}$ from transformer block 12 and compute dimension-wise difference magnitudes:

$$\Delta_i = |\text{mean}(h_{12,i}^{\text{success}}) - \text{mean}(h_{12,i}^{\text{failure}})| \quad (6)$$

Finding: The strongest probe dimensions are 121, 346, 615, 820, and 169. A logistic regression probe on these five coordinates reaches 72.5% held-out accuracy, compared with 75.0% for the full-layer probe and 67.5% for the top-two-dimensional projection used in Figure 9.

Key observations:

1. **A compact probe nearly matches the full layer:** The top-5 probe reaches 72.5% accuracy, which is 96.7% of the full-layer probe’s 75.0% accuracy.
2. **The signal is compact but not exclusive:** The same five dimensions account for only 2.51% of the total absolute mean shift, so much of the layer still moves between successful and failed generations even though a small probe captures most linear separability.
3. **Dimension 121 dominates the mean gap:** It shows a 5.73-unit difference between successful and failed generations, while the other top coordinates contribute smaller but consistent shifts.

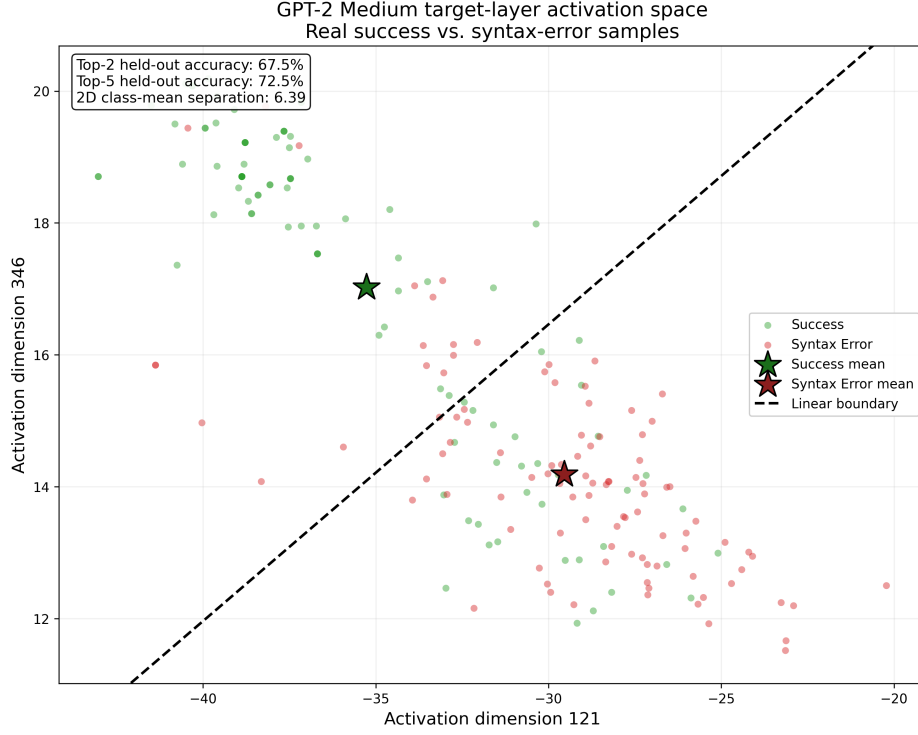


Figure 9: GPT-2 Medium layer-12 activations projected onto the top two discriminative coordinates (121, 346). Success and syntax-failure samples are partially separable, but not cleanly so: the 2D probe reaches 67.5% accuracy versus 72.5% for the top-5 probe. The figure should be read as a compact readout, not a full mechanistic decomposition.

4. **The 2D view is only partially separable:** The top-two-dimensional projection reaches 67.5% held-out accuracy with a class-mean separation of 6.39, so Figure 9 should be read as substantial but incomplete linear structure rather than a perfect boundary.

4.13. Cross-Model Comparison: Distinctive Ablation Profiles

To compare the three models more conservatively, we summarize the most informative ablation signature in each architecture rather than treating a single “critical depth” as a universal law.

Finding: Later signature layers coincide with stronger residual behavior in this three-model comparison, but we treat that pattern as descriptive rather than a scaling law. The clearest contrast is qualitative: GPT-2 Small

Table 14: Signature Ablation Layers Across Models

Model	Signature layer	Residual success	Failure mode
GPT-2 Small	Layer 2 / 17%	1.8%	98.2% syntax
GPT-2 Medium	Layer 12 / 50%	0.0%	76.1% syntax, 23.9% runtime
CodeGen	Layer 13 / 65%	29.5%	51.6% syntax, 18.9% runtime

Table 15: Scaled Ablation Pilot for GPT-2 Medium Layer 12

Scale	Success %	Syntax %	Runtime %
Baseline	6.0	94.0	0.0
0.75	8.0	92.0	0.0
0.50	2.0	98.0	0.0
0.25	0.0	100.0	0.0
0.00	0.0	72.0	28.0

has only one ablation with any surviving success, GPT-2 Medium has none, and CodeGen retains useful behavior in several layers.

Interpretation: Early signature layers, such as GPT-2 Small’s layer 2, suggest that the model commits to correctness before enough context is available. GPT-2 Medium’s mid-depth layer 12 instead appears to assemble executable structure without preserving overall fault tolerance. CodeGen’s later layer 13 combines the strongest residual success with a mixed syntax/runtime failure pattern, consistent with more distributed code-relevant computation.

4.14. Scaled Layer-12 Ablation: A Graded Intervention Pilot

Full zeroing establishes that layer 12 is special, but it does not reveal whether the layer degrades smoothly under weaker interventions. We therefore ran a smaller graded-ablation pilot on GPT-2 Medium layer 12 using 10 HumanEval problems and 5 samples per problem, scaling the layer output by $\{0.75, 0.5, 0.25, 0.0\}$.

Finding: The pilot is too small to treat the 0.75 condition as a stable improvement, but the broader pattern is informative. Partial weakening of layer 12 first drives the model toward near-total syntax collapse, whereas only full removal reproduces the runtime-error regime seen in the main hard-ablation study.

Interpretation: This suggests that layer 12 is not merely a uniformly useful block. Instead, it appears to support a thresholded execution-relevant

Table 16: Layer-12 Steering and Matched-Control Pilot

Condition	Success %	Syntax %	Runtime %
Baseline	6.0	94.0	0.0
Target -2.0	10.0	90.0	0.0
Target -1.0	2.0	94.0	4.0
Target 1.0	4.0	96.0	0.0
Target 2.0	12.0	88.0	0.0
Random mean ($n = 20$)	6.2	92.9	0.7
Random max	14.0	—	—

stage: modest attenuation disrupts syntactic coherence, while complete removal exposes a second regime in which partially executable but incorrect programs survive. That pattern is consistent with layer 12 acting as an assembly point for executable program structure rather than a generic source of accuracy.

4.15. Constructive Steering and Specificity Controls

The probe results identify a compact readout, but a stronger mechanistic question is whether the same subspace can be used constructively during generation. We therefore steer GPT-2 Medium layer 12 with the top-5 contrast vector $v = P_5(\mu_{\text{success}} - \mu_{\text{syntax}})$ and compare the result against matched sparse random controls that preserve sparsity and coefficient magnitudes but use different dimensions.

Finding: The best learned steering condition ($\alpha = 2.0$) doubles subset success from 6.0% to 12.0%. Under an expanded set of 20 matched sparse random controls, the control mean remains near baseline at 6.2%, and the learned vector beats 19 of 20 controls, but one control reaches 14.0%. We therefore treat the constructive effect as real on this subset while treating the specificity evidence as suggestive rather than conclusive.

Interpretation: The sign pattern is not perfectly monotonic: $\alpha = -2.0$ also improves success to 10.0%, while $\alpha = -1.0$ introduces a small runtime regime. We therefore interpret the top-5 vector as a *locally useful intervention axis* rather than a globally linear notion of correctness. The strongest supported claim is that the identified subspace is not merely diagnostic, because targeted intervention along it can improve execution. The control comparison still points in the same direction, but with empirical $p \leq 0.095$ it does

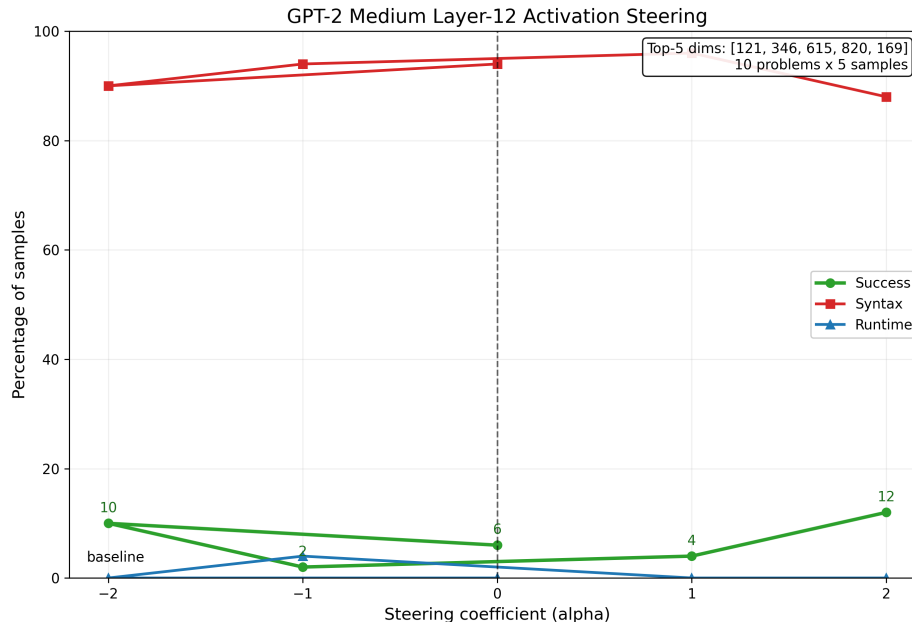


Figure 10: GPT-2 Medium layer-12 steering on the 10-problem / 5-sample subset. The best positive condition doubles success from 6.0% to 12.0%, but the response is non-monotonic and strong negative steering also helps. The subspace is therefore intervention-useful without behaving like a single signed correctness axis.

not yet justify a stronger claim of clean subspace specificity.

4.16. Scaled Layer-13 Ablation in CodeGen

To test whether the more robust code-specialized model also shows a graded signature, we ran the same 10-problem / 5-sample attenuation study on CodeGen’s strongest residual layer (layer 13).

Finding: Unlike GPT-2 Medium, CodeGen does not collapse immediately under partial attenuation. Layer 13 retains 28-30% success at scales 0.75 and 0.5, collapses to 2.0% success only at scale 0.25, and still preserves 24.0% success under full zeroing while shifting 24.0% of completions into runtime errors.

Interpretation: This is the clearest graded cross-model contrast. GPT-2 Medium layer 12 acts like a brittle thresholded checkpoint, whereas CodeGen layer 13 behaves like a partially redundant execution stage. Even when layer 13 is fully removed, surrounding layers still support a meaningful fraction

GPT-2 Medium Layer-12 Steering Is Constructive but Only Suggestively Specific

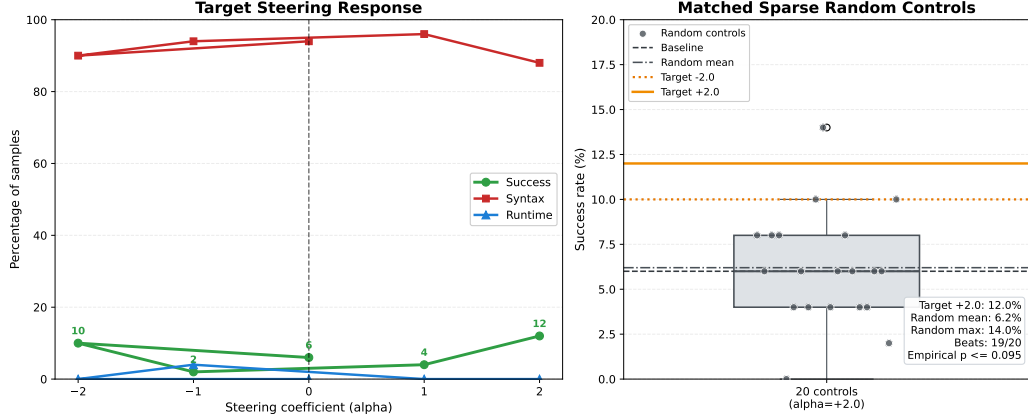


Figure 11: Learned steering versus matched sparse controls. The learned vector beats the control mean and 19 of 20 controls, but one random control still reaches 14.0% success. This supports a useful local intervention more than a clean specificity claim.

Table 17: Scaled Ablation Pilot for CodeGen Layer 13

Scale	Success %	Syntax %	Runtime %
Baseline	40.0	60.0	0.0
0.75	28.0	72.0	0.0
0.50	30.0	70.0	0.0
0.25	2.0	98.0	0.0
0.00	24.0	52.0	24.0

of executable completions. That is stronger evidence for distributed code-relevant computation than the full-zero ablation table alone.

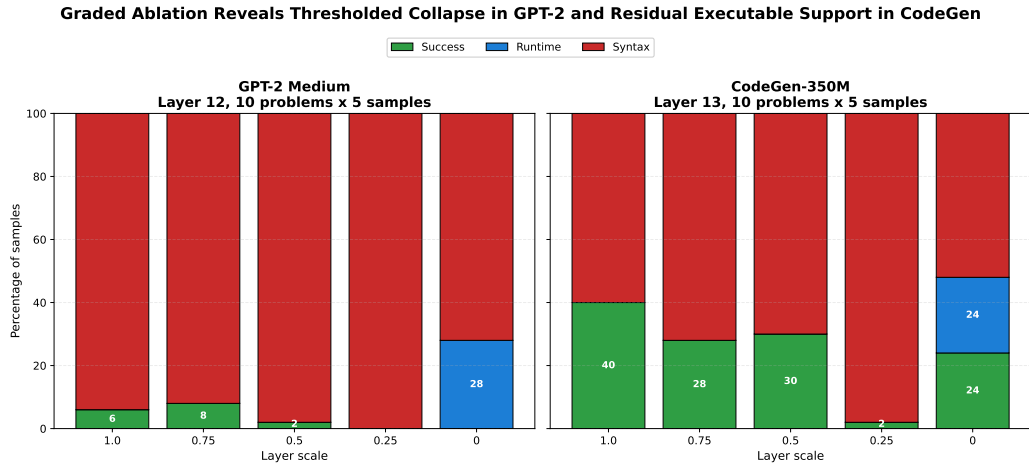


Figure 12: Matched graded-ablation comparison of GPT-2 Medium layer 12 and CodeGen layer 13. GPT-2 Medium behaves like a thresholded bottleneck: moderate attenuation drives syntax collapse and only full removal reopens runtime errors. CodeGen retains substantial success under partial attenuation and a mixed success/runtime regime even at zero, consistent with more distributed executable-code support.

5. Analysis and Discussion

5.1. Why Does Scaling Fail for GPT-2?

Our ablation results argue against a simple “larger GPT-2 should inherit more of the same computation” picture. GPT-2 Medium does not gain broader fault tolerance from added depth: every single-layer ablation drives success to 0%. Relative to GPT-2 Small, scaling from 12 to 24 layers adds capacity without adding distributed robustness, since Small still retains 1.8% success at one early layer while Medium retains none anywhere.

What changes is not residual success but the presence of a narrow execution-relevant checkpoint at layer 12. When that layer is removed, 23.9% of completions become runtime errors rather than collapsing entirely to syntax errors. No other GPT-2 Medium layer shows that behavior. The most plausible interpretation is that the medium model can assemble partially executable programs, but that computation is not distributed strongly enough across neighboring layers to survive a hard intervention.

The graded pilot sharpens the claim. Attenuating layer 12 to 0.5 or 0.25 drives the model toward near-pure syntax collapse, while only full removal reopens a 28.0% runtime-error regime. We therefore interpret layer 12 as a thresholded assembly stage for executable program structure rather than a redundant contributor that degrades smoothly.

5.2. How Does CodeGen Achieve Robustness?

CodeGen’s advantage is not that ablations stop mattering. Many single-layer zeroings still reduce success to 0%, so the intervention remains severe. The difference is structural: useful behavior survives in several layers rather than at one narrow checkpoint. Layers 5 and 7 retain nonzero success, suggesting that early syntactic support is partially distributed, and layer 13 combines the largest residual success (29.5%) with 18.9% runtime errors, marking a later executable-code regime.

The graded layer-13 pilot makes the cross-model contrast concrete. At scales 0.75 and 0.5, CodeGen still retains 28–30% success; only strong attenuation at 0.25 collapses nearly everything into syntax errors, and full zeroing reopens a mixed 24.0% success / 24.0% runtime regime. The most economical interpretation is *partial* redundancy: code-relevant computation is spread across neighboring layers even though the model remains vulnerable to severe interventions.

5.3. What the CodeGen Ladder Adds

The within-family CodeGen ladder removes the strongest family-difference objection. GPT-2 versus CodeGen alone leaves open tokenization, optimizer, and implementation confounds; NL→Multi→Mono keeps architecture and parameter count fixed while changing continued-pretraining data and curriculum.

The ladder still does *not* support an overclean law. HumanEval is not monotone because Multi is slightly worse than NL, so we do not claim a matched-compute or data-only rule. The supported claim is narrower and stronger: within a fixed 350M architecture, later code- and Python-focused continued pretraining materially improves the best checkpoint, and MBPP shows a fully monotone NL<Multi<Mono progression.

The scaled follow-up strengthens that interpretation without pretending the whole ladder is mechanistically tidy. NL layer 11 behaves like brittle syntax support, Mono layer 13 reopens the mixed runtime regime seen in the main CodeGen model, and Multi remains anomalous under full zeroing. The safest conclusion is therefore that continued pretraining changes both benchmark performance and the geometry of residual code support, even if it does not induce a simple monotone law at every intermediate checkpoint.

5.4. Implications for Model Design

Our findings suggest three architectural principles for robust code generation:

- 1. Distribute processing across depth:** Robust models should preserve some residual success or at least diverse failure signatures under harsh interventions, rather than collapsing uniformly at every layer.

- 2. Track failure-mode transitions, not just success collapse:** Layer 12 in GPT-2 Medium is informative precisely because it changes the balance of syntax vs. runtime errors. The new scaled-ablation pilot suggests that this transition is thresholded: syntax collapses first, and the runtime regime appears only after complete removal.

- 3. Code-specific continued pretraining matters:** The GPT-2 checkpoints trained on WebText do not show the same residual robustness profile as CodeGen under our interventions. Code-specific pretraining (The Pile + GitHub) Gao et al. (2021); Nijkamp et al. (2022) is consistent with broader residual robustness under the same intervention, and the within-family CodeGen ladder shows that later code- and Python-focused continued-pretraining

stages can improve behavior even without changing architecture or parameter count.

5.5. Implications for Evaluation Practice

The targeted robustness controls add a practical lesson for empirical studies of code models: benchmark scores are partly properties of the surrounding generation protocol, not only of the checkpoint. In our saved-artifact controls, prompt format substantially changes HumanEval sample success for the code-specialized models, and MBPP low-temperature decoding increases repeated successful samples while reducing the number of distinct tasks solved at least once.

For software-engineering evaluations, this argues for reporting more than a single aggregate pass rate whenever the goal is reliability. Prompt form, strict-test sensitivity, and task-level coverage should be documented alongside sample-level success so that an apparent gain is not mistaken for broader problem coverage or stronger executable correctness.

5.6. What the LiveCodeBench Result Changes

LiveCodeBench sharpens the scope of our claim more than the ranking itself. On HumanEval and MBPP, CodeGen’s advantage over GPT-2 is large and practically meaningful. On LiveCodeBench release_v2, all three small models are near-zero, with CodeGen retaining only a marginal nonzero signal. The right framing is therefore not that small code-pretrained models are broadly strong programmers, but that code-specific pretraining improves classical Python synthesis performance and internal robustness without closing the gap to harder modern coding tasks.

This harsher benchmark is still scientifically useful. It reinforces the negative result because GPT-2 Small and GPT-2 Medium remain indistinguishable at zero, while also preventing us from overclaiming from HumanEval or MBPP alone.

5.7. Why a Small Probe Still Works

The five-dimensional probe works because it captures most of the linear separability available in this layer: 72.5% held-out accuracy versus 75.0% for the full 1024-dimensional probe. In that operational sense, a very small coordinate set is enough for a strong readout of success versus syntax failure.

That does *not* mean that five neurons implement correctness. The top five dimensions account for only 2.51% of the total absolute mean shift, so the broader representation remains distributed.

The steering results sharpen the interpretation. The best positive condition doubles success, but one random control still does better and the response curve is non-monotonic. We therefore treat the top-5 subspace as a useful local handle on execution behavior rather than a complete mechanistic decomposition or a signed “correctness-neuron” explanation. These dimensions remain promising targets for monitoring and future interventions Meng et al. (2022), but not a complete description of the underlying algorithm.

5.8. Threats to Validity

Construct validity: We evaluate executable correctness with benchmark tests. This is appropriate for studying code-generation reliability, but it does not measure maintainability, readability, security, or integration quality. We therefore frame the results as short-form Python synthesis and execution robustness, not as a complete theory of software engineering ability.

Internal validity: The main comparisons depend on fixed decoding settings, saved artifacts, and executor behavior. We reduce this risk by reporting exact sample counts, using problem-level bootstrap tests, separating sample-level and task-level conclusions, and recording the HumanEval/129 coverage caveat for the synced CodeGen artifact. The smaller intervention pilots are explicitly treated as causal probes rather than full-benchmark replacements.

External validity: The study covers small decoder-only models, mostly Python tasks, and pretrained checkpoints. The conclusions should not be read as claims about frontier code models, instruction-tuned assistants, repository-level repair, or production deployment. The CodeGen ladder provides a stronger within-family control, but it is still not a matched-compute causal isolation of pretraining data.

Conclusion validity: Several results are intentionally framed conservatively. The GPT-2 Small versus Medium HumanEval gap is numerically small and not statistically reliable. The steering result improves subset success, but one matched random control outperforms the learned vector, so we claim a locally useful intervention axis rather than a definitive sparse mechanism. This conservative framing is part of the evidentiary standard rather than a rhetorical limitation.

5.9. Limitations and Future Work

The validity threats above imply several direct extensions:

1. Benchmark scope is broader but still limited: We evaluate on HumanEval, HumanEval+, MBPP, MBPP+, and contamination-aware LiveCodeBench Chen et al. (2021); Austin et al. (2021); Liu et al. (2023); Jain et al. (2024), which is stronger than the earlier two-benchmark version, but all settings still emphasize short-form code-generation tasks rather than long-horizon repository-level software engineering.

2. Pretrained models only: We do not evaluate fine-tuned models. While our goal was to isolate base architecture effects, fine-tuning could potentially repair GPT-2 Medium’s bottleneck by redistributing computation across layers.

3. The CodeGen ladder is controlled but not matched-compute: NL, Multi, and Mono hold architecture and parameter count fixed, but they also differ in additional tokens, optimization steps, and curriculum. The ladder therefore supports a strong within-family continued-pretraining claim, not a perfectly factorized causal isolation of data distribution alone.

4. Graded interventions remain limited: We include scaled-ablation pilots for GPT-2 Medium layer 12, CodeGen layer 13, and a final ladder follow-up on CodeGen-NL / Multi / Mono signature layers. Even so, these remain small targeted subsets rather than full benchmark sweeps, and one ladder leg (CodeGen-Multi layer 7) behaved anomalously under full zeroing. Broader graded sweeps remain necessary before claiming a general law of thresholded behavior.

5. Limited semantic analysis: We identify which layers validate syntax but do not deeply analyze which layers perform semantic reasoning (variable scoping, type consistency, algorithm correctness). Future work should extend our methods to semantic error detection.

6. Constructive intervention evidence is still small-scale: We observe positive rescue from learned steering on a subset, and the learned vector beats 19 of 20 matched sparse random controls, but one control does better and the empirical control test remains only suggestive ($p \leq 0.095$). Larger control sets and cross-benchmark steering experiments are needed before treating the subspace as a robust causal handle.

Despite these limitations, our core finding is that code models differ not only in accuracy but in their layerwise failure profiles under intervention, providing actionable insight for model design and evaluation.

5.10. Reproducibility and Artifact Availability

This paper is intended to be artifact-traceable rather than narrative-only. The quantitative claims in the main text are backed by saved processed outputs under `outputs/tables/`, benchmark result files under `data/results_*/`, figure-generation scripts under `scripts/`, and the claim-to-artifact audit included with the review package. In particular, the benchmark-summary figures are regenerated from saved bootstrap summaries, strict-evaluation summaries, LiveCodeBench summaries, and repaired CodeGen ladder artifacts rather than from hand-copied spreadsheet values.

For JSS review, the intended submission package includes a supplementary ZIP containing the processed tables, benchmark result artifacts, analysis scripts, source code, configuration files, plotting scripts, figures, and audit reports needed to trace the reported numbers. Raw local execution logs are preserved in the working repository but are not required for the reviewer-facing package because the reported claims are checked against the structured summaries and artifacts. A public archival repository can be created from this curated archive after the review-facing version is accepted or explicitly requested. We keep the main-paper coverage note for `HumanEval/129` explicit because reproducibility includes honest accounting of what was and was not saved in the synced main CodeGen artifact.

5.11. Broader Impact

The immediate positive impact of this work is methodological. Better tools for diagnosing where code models fail can improve evaluation practice, reduce overclaiming from small benchmark gains, and help future systems distribute executable-code behavior more robustly across depth. The strongest practical message is not that small models are already good programmers, but that pretraining choices and failure-mode analysis deserve at least as much attention as raw scale in this regime.

The main risk is dual use. Mechanistic insight into code-generation bottlenecks, especially if paired with constructive steering or future stronger interventions, could eventually be used to make automated code generation more capable for both beneficial and harmful ends, including insecure code production or malicious automation. We partially mitigate that risk by studying small openly available models, by emphasizing reliability analysis over deployment recipes, and by treating the steering results as limited scientific evidence rather than as a production-ready capability-improvement method.

6. Conclusion

We show that, in this small-model regime, the conventional expectation that “bigger models are better” does not hold reliably for code generation: GPT-2 Medium (355M parameters) fails to improve meaningfully over GPT-2 Small (124M parameters) despite $2.9\times$ more parameters.

Problem-level bootstrap analysis makes this conclusion more precise: GPT-2 Small and GPT-2 Medium are statistically indistinguishable on HumanEval in our setting, whereas both remain more than 30 percentage points behind CodeGen-350M with highly significant pairwise differences.

The same claim transfers to full-coverage MBPP. Across 257 tasks with 20 samples each, GPT-2 Small and GPT-2 Medium both achieve 0.0% success, whereas CodeGen-350M reaches 7.39% success and 22.96% pass@5. The pairwise MBPP success-rate gaps against CodeGen remain highly significant ($p = 0.0001$), while the GPT-2 Small vs. GPT-2 Medium difference vanishes entirely ($p = 1.0$).

A stricter HumanEval+ re-evaluation preserves the same ordering. On the first five saved samples per task, GPT-2 Small and GPT-2 Medium both remain at 0.0% pass@1, whereas CodeGen-350M retains 2.1% pass@1 after the augmented hidden tests.

The same conclusion survives a separate strict MBPP+ evaluation with EvalPlus. On 378 tasks scored under the stronger Base+Extra metric, GPT-2 Small and GPT-2 Medium both remain at 0.0% pass@1 and pass@10, whereas CodeGen-350M retains 1.1% pass@1 and 6.4% pass@10.

The contamination-aware LiveCodeBench result narrows and strengthens the paper at the same time. GPT-2 Small and GPT-2 Medium remain at 0.0% on pass@1, pass@5, and pass@10, while CodeGen-350M remains only marginally above zero (0.02%, 0.10%, and 0.20%). Thus, the strongest positive comparative result remains concentrated on HumanEval and MBPP, but the negative conclusion that scale alone is insufficient survives even under a much harder modern benchmark.

The within-family CodeGen ladder sharpens the paper further. At fixed 350M scale, CodeGen-Mono is strongest on both HumanEval and MBPP, while the MBPP ladder is cleanly monotone from NL to Multi to Mono and the HumanEval ladder is full-coverage but mixed at the intermediate Multi checkpoint. Because the checkpoints receive additional compute and curriculum, we interpret this not as a pure data-only law but as strong within-family evidence that later code- and Python-focused continued pretraining

materially improves small-model code generation.

Under full-layer ablation, GPT-2 Medium never preserves success; instead, layer 12 stands out because its removal uniquely shifts 23.9% of failures into runtime errors while every other layer collapses almost entirely to syntax errors.

A graded layer-12 pilot strengthens this interpretation. Partial attenuation first pushes GPT-2 Medium toward complete syntax collapse, while only full zeroing reopens a 28.0% runtime-error regime. This suggests that layer 12 marks a thresholded execution-relevant stage rather than a smoothly redundant computation.

In contrast, CodeGen-350M Nijkamp et al. (2022) still shows severe sensitivity to hard ablation, but it exhibits broader residual robustness than either GPT-2 variant. Four layers retain nonzero success, and the graded layer-13 pilot preserves 28-30% success under partial attenuation and 24.0% success even under full zeroing, indicating that code-relevant computation is less concentrated than in GPT-2. The final ladder follow-up is consistent with that picture at the two clean checkpoints: CodeGen-NL layer 11 degrades toward syntax collapse under attenuation, whereas CodeGen-Mono layer 13 again preserves a mixed executable/runtime regime under full zeroing.

Our activation analysis shows that a compact probe on five layer-12 dimensions (121, 346, 615, 820, 169) reaches 72.5% held-out accuracy, close to the 75.0% achieved by a full-layer linear probe. More importantly, constructive steering along this same sparse direction doubles subset success from 6.0% to 12.0%. The expanded random-control comparison is favorable but not decisive, so the safest conclusion is that this direction is a usable local control subspace rather than a purely descriptive probe.

The targeted robustness controls add a systems-level caution to that mechanistic claim. Prompt format and decoding temperature can substantially change sample-level success, and on MBPP low-temperature decoding improves repeated sample success while reducing distinct-task coverage. Thus, the bottleneck is partly internal to the model and partly induced by the generation/evaluation protocol around it.

These findings challenge the scaling-centric paradigm in neural code generation Kaplan et al. (2020); Wei et al. (2022). Parameter count alone does not determine capability. Across HumanEval, HumanEval+, MBPP, MBPP+, and a harsher contamination-aware LiveCodeBench stress test, the central predictor of useful code behavior in this small-model regime is not scale within the GPT-2 family, but whether pretraining has already distributed

code-relevant computation across depth. The within-family CodeGen ladder makes this point harder to dismiss: even with architecture and parameter count held fixed, later continued-pretraining stages can materially change both performance and the layerwise organization of executable-code behavior. For practitioners, the main design lessons are to distribute code-relevant processing across depth, analyze failure-mode shifts under intervention, and use code-specific pretraining to build residual robustness.

Future work should extend our mechanistic analysis to fine-tuned models, multilingual code generation, and semantic error detection, with the ultimate goal of reverse-engineering the complete algorithms by which neural networks generate syntactically and semantically correct programs.

Data Availability

The processed result tables, plotting scripts, benchmark summaries, saved result artifacts, configuration files, source code, and claim-to-artifact audit used in this manuscript are available in the curated supplementary artifact archive submitted with this manuscript. Large model checkpoints and raw model caches are not redistributed; all experiments use public pretrained checkpoints and regenerate derived outputs from the provided scripts and configuration files.

Declaration of Competing Interest

The author declares no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of Generative AI and AI-Assisted Technologies in the Manuscript Preparation Process

During the preparation of this work, the author used OpenAI Codex to assist with manuscript editing, code execution orchestration, artifact organization, and formatting. After using this tool, the author reviewed and

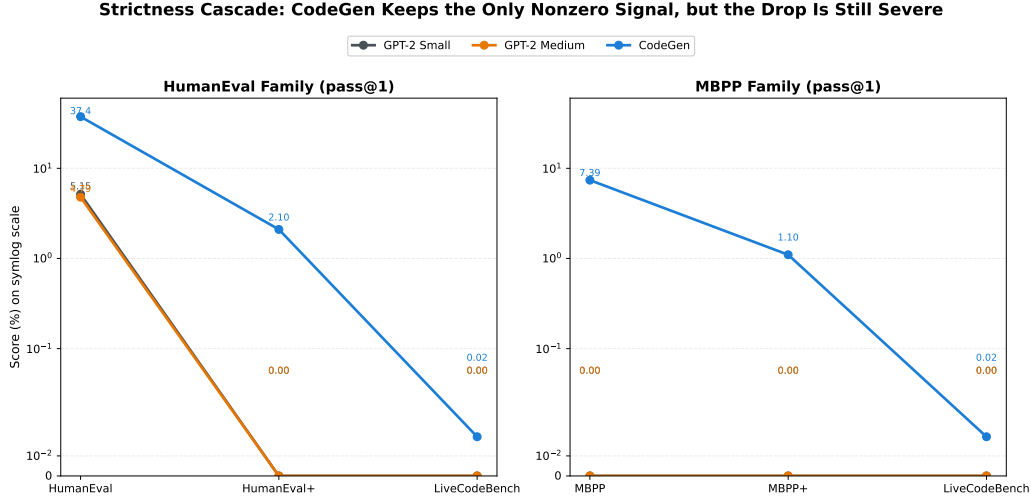


Figure A.13: Pass@1 cascade across benchmark families. CodeGen retains the only nonzero signal as evaluation becomes stricter, but the drop from classical benchmarks to HumanEval+/MBPP+ and LiveCodeBench is severe. GPT-2 Small and GPT-2 Medium remain indistinguishable at or near zero throughout.

edited the content as needed and takes full responsibility for the content of the submitted manuscript.

Appendix A. Supplementary Benchmark and Provenance Figures

The main paper emphasizes the benchmark-wide result map, bootstrap intervals, and within-family CodeGen ladder. We place two additional supporting figures here for transparency: a pass@1 strictness cascade across benchmark families and a provenance-oriented coverage audit for the repaired HumanEval artifacts.

References

- Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q., Sutton, C., 2021. Program synthesis with large language models. arXiv preprint arXiv:2108.07732 .
- Chen, M., Tworek, J., Jun, H., Yuan, Q., de Oliveira Pinto, H.P., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B.,

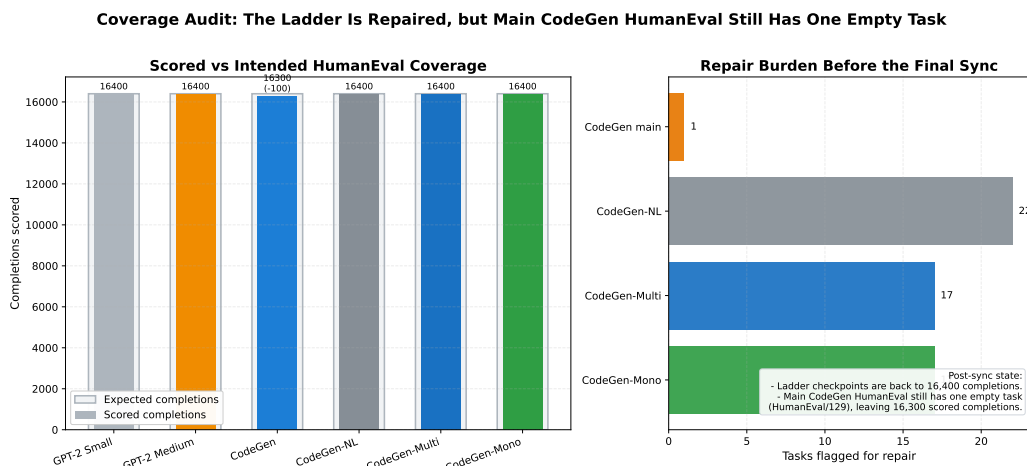


Figure A.14: Coverage and repair audit for the final HumanEval artifacts. The repaired CodeGen ladder checkpoints return to full 16,400-completion coverage, while the synced main CodeGen HumanEval artifact still contains one empty task (HumanEval/129), leaving 16,300 scored completions. The right panel records the pre-sync repair burden that produced the final ladder artifacts.

Gray, S., Ryder, N., Pavlov, M., Power, A., Kaiser, L., Bavarian, M., Winter, C., Tillet, P., Such, F.P., Cummings, D., Plappert, M., Chantzis, F., Barnes, E., Herbert-Voss, A., Guss, W.H., Nichol, A., Paino, A., Tezak, N., Tang, J., Babuschkin, I., Balaji, S., Jain, S., Saunders, W., Hesse, C., Carr, A.N., Leike, J., Achiam, J., Misra, V., Morikawa, E., Radford, A., Knight, M., Brundage, M., Murati, M., Mayer, K., Welinder, P., McGrew, B., Amodei, D., McCandlish, S., Sutskever, I., Zaremba, W., 2021. Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374 .

Clark, K., Khandelwal, U., Levy, O., Manning, C.D., 2019. What does bert look at? an analysis of bert’s attention. arXiv preprint arXiv:1906.04341 .

Conmy, A., Mavor-Parker, A.N., Lynch, A., Heimersheim, S., Garriga-Alonso, A., 2023. Towards automated circuit discovery for mechanistic interpretability. Advances in Neural Information Processing Systems 36.

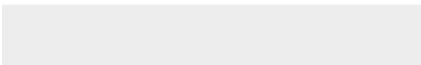
Gao, L., Biderman, S., Black, S., Golding, L., Hoppe, T., Foster, C., Phang, J., He, H., Thite, A., Nabeshima, N., et al., 2021. The pile: An 800gb dataset of diverse text for language modeling. arXiv preprint arXiv:2101.00027 .

- Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., Casas, D.d.L., Hendricks, L.A., Welbl, J., Clark, A., et al., 2022. Training compute-optimal large language models. arXiv preprint arXiv:2203.15556 .
- Jain, N., Han, K., Gu, A., Li, W.D., Yan, F., Zhang, T., Wang, S., Solar-Lezama, A., Sen, K., Stoica, I., 2024. Livecodebench: Holistic and contamination free evaluation of large language models for code. arXiv preprint arXiv:2403.07974 .
- Kaplan, J., McCandlish, S., Henighan, T., Brown, T.B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., Amodei, D., 2020. Scaling laws for neural language models. arXiv preprint arXiv:2001.08361 .
- Li, Y., Choi, D., Chung, J., Kushman, N., Schrittwieser, J., Leblond, R., Eccles, T., Keeling, J., Gimeno, F., Dal Lago, A., et al., 2022. Competition-level code generation with alphacode. *Science* 378, 1092–1097.
- Liu, J., Xia, C.S., Wang, Y., Zhang, L., 2023. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. arXiv preprint arXiv:2305.01210 .
- Ma, W., Liu, S., Zhao, M., Xie, X., Wang, W., Hu, Q., Zhang, J., Liu, Y., 2024. Unveiling code pre-trained models: Investigating syntax and semantics capacities. arXiv preprint arXiv:2212.10017 .
- Meng, K., Bau, D., Andonian, A., Belinkov, Y., 2022. Locating and editing factual associations in gpt. *Advances in Neural Information Processing Systems* 35, 17359–17372.
- Nijkamp, E., Pang, B., Hayashi, H., Tu, L., Wang, H., Zhou, Y., Savarese, S., Xiong, C., 2022. Codegen: An open large language model for code with multi-turn program synthesis. arXiv preprint arXiv:2203.13474 .
- Olsson, C., Elhage, N., Nanda, N., Joseph, N., DasSarma, N., Henighan, T., Mann, B., Askell, A., Bai, Y., Chen, A., Conerly, T., Drain, D., Ganguli, D., Hatfield-Dodds, Z., Hernandez, D., Johnston, S., Jones, A., Kernion, J., Lovitt, L., Ndousse, K., Amodei, D., Brown, T., Clark, J., Kaplan, J., McCandlish, S., Olah, C., 2022. In-context learning and induction heads. arXiv preprint arXiv:2209.11895 .

- Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., Sutskever, I., et al., 2019. Language models are unsupervised multitask learners. OpenAI blog 1, 9.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I., 2017. Attention is all you need. Advances in Neural Information Processing Systems 30.
- Vig, J., Gehrmann, S., Belinkov, Y., Qian, S., Nevo, D., Singer, Y., Shieber, S., 2020. Investigating gender bias in language models using causal mediation analysis. Advances in Neural Information Processing Systems 33, 12388–12401.
- Voita, E., Talbot, D., Moiseev, F., Sennrich, R., Titov, I., 2019. Analyzing multi-head self-attention: Specialized heads do the heavy lifting, the rest can be pruned. arXiv preprint arXiv:1905.09418 .
- Wei, J., Tay, Y., Bommasani, R., Raffel, C., Zoph, B., Borgeaud, S., Yogatama, D., Bosma, M., Zhou, D., Metzler, D., et al., 2022. Emergent abilities of large language models. Transactions on Machine Learning Research .



Click here to access/download
LaTeX Source Files
jss_source_package.zip



Declaration of Interest Statement

The author declares no competing interests.